

浙江大学第二十三届 大学生数学建模竞赛

2025 年 5 月 9 日—5 月 19 日

团队编号 8447

题 目 A ☒ B

(在所选题目上打勾)

	参赛队员 1	参赛队员 2	参赛队员 3
姓名	徐煜岑	叶凯森	林彦名
学号	3230103471	3230100127	3230102883
院(系)	电气工程学院	生物系统工程与 食品科学学院	电气工程学院
专业	电气工程及其自动化	农业工程	电气工程及其自动化
手机	13757062526	15857867937	13666561580
Email	3230103471@zju.edu.cn	3230100127@zju.edu.cn	3230102883@zju.edu.cn

浙江大学本科生院
浙江大学数学建模实践基地

分布式能源配电网风险分析

叶凯森，林彦名，徐煜岑

2025 年 5 月 16 日

摘要

本文目标为量化分析分布式能源（DG）接入对配电网运行风险的影响，包括失负荷风险与过负荷风险。论文构建了一个基于概率建模与网络拓扑分析的综合框架，并以一个 62 节点有源配电网作为典型场景进行案例研究。

研究将系统总风险定义为失负荷风险和过负荷风险的加权和。针对问题一，模型通过蒙特卡洛模拟生成多种故障场景，结合动态功率分配策略和非线性用户危害度函数来量化失负荷风险。同时，过负荷风险的评估则聚焦于 DG 出力波动对馈线电流的影响，并采用经济性优化（最小化 DG 削减成本）与物理安全性验证（潮流计算确保线路不越限）相结合的双阶段策略。

针对问题二，研究引入 DG 容量因子 (γ)，系统地分析了 DG 接入容量从一倍逐步提升至三倍时，系统总风险、失负荷风险和过负荷风险的演变规律。通过对不同容量因子下的众多随机场景进行模拟计算，得到期望风险值。

模拟结果显示，随着 DG 容量因子的增加：过负荷风险成本显著降低，当容量因子达到 2.5 及以上时，该风险几乎可以忽略不计。失负荷风险成本呈现非单调变化趋势，在 DG 容量中等增加的阶段可能略有上升，但当容量达到较高水平后，失负荷风险显著下降。系统总风险主要由失负荷风险主导，总体上随着 DG 容量的增加而降低，在本研究的参数范围内，当容量因子达到 3.0 时，系统总风险最低。

本文还对模型的关键参数进行了灵敏度分析，发现线路单位长度故障率对系统总风险的影响最为显著。同时，讨论了模型存在的误差来源，如输入数据不确定性、模型结构简化以及蒙特卡洛模拟的收敛性问题。

模型的优点在于构建了较为全面的风险评估框架，能够有效处理不确定性，并具有较好的参数化和灵活性。不足之处主要包括潮流计算的简化、DG 出力与负荷模型的简化等。未来研究可从集成精确潮流计算模块、完善不确定性来源建模、深化元件故障与恢复模型等方面进行推广。

本研究为高比例分布式能源接入背景下配电网的风险评估与规划运行提供了理论支撑和决策依据，具有较高价值。

关键词：分布式能源，配电网，风险评估，蒙特卡洛算法，联络开关

目录

1 问题重述	3
1.1 问题背景	3
1.2 问题提出	3
2 问题分析	3
3 模型假设与符号说明	4
3.1 模型基本假设	4
3.2 符号说明	4
4 模型建立与求解	8
4.1 问题 1 模型建立与求解	8
4.1.1 问题 1 求解思路	8
4.1.2 问题 1 模型建立	8
4.1.3 问题 1 模型求解与分析	11
4.2 问题 2 模型建立与求解	11
4.2.1 问题 2 求解思路	11
4.2.2 问题 2 模型建立	12
4.2.3 问题 2 模型求解与分析	14
5 模型分析检验	17
5.1 灵敏度分析	17
5.2 误差分析	18
6 模型评价与推广	20
6.1 模型的优点	20
6.2 模型的不足	20
6.3 模型的推广	21
A 附录	24

1 问题重述

1.1 问题背景

随着我国“双碳”战略目标的推进，可再生能源的规模化开发与高效利用成为能源转型的核心路径。分布式能源（Distributed Generation, DG）因其清洁性与就地消纳优势，在配电网中的渗透率持续提升。然而，DG 的间歇性、波动性及双向功率流动特性，对传统配电网的运行模式、安全控制及风险评估提出了全新挑战。尤其在“双高”（高比例新能源、高电力电子化）背景下，如何量化分析 DG 接入引发的失负荷与过负荷风险，已成为配电网规划与运行的关键难题。

配电网作为连接用户与主网的“最后一公里”，其可靠性直接影响社会经济活动与居民生活质量。故障导致的失负荷需通过联络线转供实现复电，而供电恢复的核心目标是在拓扑重构中最小化经济损失。由于用户类型（居民住宅、商业、政府机构等）对停电损失的敏感性差异显著，需建立基于危害度函数的差异化评估体系，精准刻画不同场景下的经济损失。与此同时，DG 的接入改变了馈线潮流分布：其出力的不确定性可能导致局部过负荷，而“禁止倒送功率”的运行约束又限制了冗余功率的消纳路径，进一步加剧了系统调控压力。

1.2 问题提出

当前，风险评估多采用概率 - 后果框架（如公式 $R_{\text{sys}} = P_{\text{LL}}C_{\text{LL}} + P_{\text{OL}}C_{\text{OL}}$ ），但现有研究在以下方面仍存不足：（1）失负荷模型多假设转供能力为理想值，未充分考虑联络线容量及相邻馈线负载状态；（2）过负荷分析常忽略 DG 出力波动与馈线间功率调节的耦合效应；（3）危害度函数多采用线性损失模型，难以反映短时极端停电或过载的非线性经济影响。针对上述问题，亟需构建兼顾网络拓扑、运行约束及多源不确定性的精细化风险评估模型，并通过实际系统验证模型的适应性与有效性。

本研究以 62 节点有源配电网为典型场景，围绕 DG 容量递增下的风险演变规律开展分析，旨在揭示分布式能源高渗透对系统安全与经济性的动态影响机制，为新型电力系统的韧性提升与运行优化提供理论支撑。

2 问题分析

分布式能源接入配电网的风险评估需围绕失负荷与过负荷两类核心风险展开，其核心在于通过数学建模量化故障事件的发生概率及其危害程度，并分析风险随分布式能源容量变化的演变规律。风险评估框架以概率与危害的乘积之和为基础，需从故障场景生成、网络拓扑分析及危害度函数构建三方面入手：一方面需结合线路、开关、分布式能源及负荷的独立故障率，通过蒙特卡洛模拟生成海量故障场景，统计失负荷与过负荷事件的频率；另一方面需根据用户类型差异及停电持续时间，建立非线性危害度函数，体现经济损失的动态特性，同时满足“分布式能源不得向上级电网倒送功率”的运行约束。

失负荷风险建模需重点解决故障后供电恢复的动态转供问题。需基于网络拓扑识别故障影响范围，结合相邻馈线的剩余容量及联络线传输能力，动态分配失电负荷的转供比例，避免高估转供能力。例如，某馈线发生故障后，需优先调用相邻馈线的可用容量，但受限于联络线物理

上限, 实际转供量取两者较小值. 最终失负荷量由故障负荷与实际转供量的差值决定, 并需结合用户类型与中断时长评估经济损失.

过负荷风险则需聚焦分布式能源出力波动对馈线电流的影响. 需在满足“不得倒送功率”约束下, 通过潮流计算验证馈线电流是否超过额定值的 10%, 并量化超额电流对设备损坏与系统稳定性的危害. 分布式能源容量递增会加剧馈线功率波动, 需通过动态调整相邻馈线间的功率转移能力, 评估过载概率与危害的变化趋势.

风险演变分析需揭示分布式能源容量从初始值逐步提升至三倍过程中, 系统风险的动态变化. 需结合蒙特卡洛模拟与潮流计算, 量化容量递增对失负荷概率、过载概率及其危害的耦合影响, 最终输出系统风险的演变曲线. 模型创新性体现在动态转供分配机制与非线性危害度函数的设计上, 但受限于故障率、负荷分布等参数的准确性, 需结合实际数据校准以提高实用性.

3 模型假设与符号说明

3.1 模型基本假设

1. 故障独立性与单一性: 各类故障(线路、开关、DG、负荷)相互独立发生, 且同一时间同一类型单元仅发生一次故障, 忽略多故障耦合场景.

2. 网络拓扑简化: 配电网运行方式设定为单电源供电模式, 即同一时刻每条馈线仅由一个变电站出线开关供电, 忽略多源协同供电的复杂拓扑. 联络开关仅用于故障后功率转移, 不参与正常运行状态下的功率调节.

3. 功率流动约束: 分布式能源(DG)出力不得向上级电网倒送, 但允许在相邻馈线间调节, 以平衡本地负荷需求. 风险计算仅考虑有功功率和电流影响, 忽略无功功率与电压越限对系统的影响.

4. 参数确定性与静态性: 线路、开关、DG及负荷的故障率设定为固定值(如线路故障率 = 长度 $\times 0.002/\text{km}$, 开关故障率 0.2%), 未考虑环境、老化等因素对故障率的动态影响. 馈线额定负载(2.2 MW)与联络线容量视为恒定参数, 未建模负载随季节、天气等因素的波动性.

5. 转供能力理想化: 相邻馈线剩余容量计算基于“馈线额定容量 - (本地负荷 - DG出力)”, 假设DG出力与负荷需求为恒定值, 未考虑其随机性对剩余容量的动态影响. 联络线容量为固定上限(如 2.2 MW), 但实际中联络线容量可能受限于设备状态或实时负载水平.

6. 风险量化范围局限性: 模型聚焦于失负荷与过负荷两类风险, 未考虑其他潜在风险(如电压暂态、谐波污染等). 危害度函数仅体现经济损失, 未包含社会影响(如重要用户供电中断引发的社会稳定性问题).

7. 计算方法简化: 蒙特卡洛模拟中DG出力与负荷需求采用随机抽样, 但未引入时间序列相关性(如光伏出力与负荷的昼夜波动耦合关系). 潮流计算简化为电流与功率的线性关系, 未严格基于三相潮流方程求解.

3.2 符号说明

表 1: 符号说明

符号	含义
R_{sys}	系统总风险值
P_{LL}	失负荷概率
P_{OL}	过负荷概率
C_{LL}	失负荷危害程度
C_{OL}	过负荷危害程度. (问题 1 中可包括经济削减成本和物理过载危害; 问题 2 中主要指物理过载危害)
$L_{\text{LL},i}$	(问题 1) 场景 i 的最终失负荷量
R_{LL}	(问题 1) 失负荷风险 (一般概念, $P_{\text{LL}} \times C_{\text{LL}}$)
$C_{\text{LL},e}$	(问题 1) 事件 e 导致的失负荷危害度
L_{fault}	(问题 1) 故障导致的总失负荷量
$F(L_j, j, t_j)$	(问题 1) 危害度函数 (用户类型 j , 失负荷功率 L_j , 停电时长 t_j)
$C_L(f_k)$	(问题 1) 故障 f_k 的失负荷危害度
$L_{\text{final_lost}}(f_k)$	(问题 1) 故障 f_k 下最终失负荷的用户集合
L_j	(问题 1) 用户类型 j 的失负荷功率
t_j	用户类型 j 的停电时长
a_j, b_j, c_j	(问题 1) 用户类型 j 相关的非线性客户停电损失函数系数
$t_{\text{crit},j}$	(问题 1) 用户类型 j 的关键停电时长阈值
$C_{\text{LL},s}(\gamma)$	(问题 2) 场景 s 、容量因子 γ 下的总失负荷危害度/损失
$R_{\text{LL}}(\gamma)$	(问题 2) 容量因子 γ 下的期望失负荷风险 (成本)
$L_{j,s}^{\text{unserved}}(\gamma)$	(问题 2) 节点 j 在场景 s 、容量因子 γ 下的最终未服务负荷
$\mathcal{N}_{\text{unserved}}^s$	(问题 2) 场景 s 中未被服务的节点集合
$L_{\text{island_initial_unserved},s}(\gamma)$	(问题 2) 孤岛在场景 s 、 γ 下的初始未满足负荷
$L_{\text{island_final_unserved},s}(\gamma)$	(问题 2) 孤岛在场景 s 、 γ 下的最终未满足负荷

表 2: 符号说明 (续表)

符号	含义
R_{OL}	(问题 1) 系统过负荷经济风险 (来自 DG 削减)
$C_{OL,i}$	(问题 1) 馈线 i 的期望削减损失
$c_{curtail,i}$	(问题 1) 馈线 i 的单位削减成本
$P_{curtail,i}$	(问题 1) 馈线 i 需本地削减的功率
c_k	(问题 1) 线路 k 的单位过载成本系数
$C_{OL,s}(\gamma)$	(问题 2) 场景 s 、容量因子 γ 下的总物理过载危害度/损失
N_{sim} (原 N)	蒙特卡洛模拟总次数
$\mathbb{I}(\cdot)$	指示函数 (条件成立为 1, 否则为 0)
$\mathbb{E}[\cdot]$	期望值
$(\cdot)^+$	取正值函数 (即 $\max(0, \cdot)$)
$\text{Beta}(\alpha, \beta)$	(问题 2) Beta 分布及其参数
G_s	(问题 2) 故障后场景 s 的网络拓扑
$\mathcal{I}_s$	(问题 2) 场景 s 中与有效电源点失去连接的孤岛集合
$\mathcal{F}_s$	(问题 2) 场景 s 中的故障负荷集合
$\mathcal{L}$	(问题 1) 过载线路集合
$\mathcal{O}_s$	(问题 2) 场景 s 中发生过载的线路集合
$\mathcal{T}_i$	(问题 1) 馈线 i 的联络线集合
$\mathcal{P}_{island}$	(问题 2) 可向孤岛转供的路径集合
γ	(问题 2) DG 容量因子
$P_{dg,k}^{base}$	(问题 2) DG 单元 k 的初始基准容量
$P_{dg,k}^{cap}(\gamma)$	(问题 2) DG 单元 k 在容量因子 γ 时的接入容量
$P_{dg,i}$ (或 DG_j)	(问题 1) 馈线 i (或 j) 的 DG 出力
$P_{dg,k}^{out}(s, \gamma)$	(问题 2) DG 单元 k 在场景 s 、 γ 下的实际出力
$P_{dg,f'}^{out}(s, \gamma)$	(问题 2) 馈线 f' 内部 DG 在场景 s 、 γ 下的实际出力
$\text{OpStatus}_k(s)$	(问题 2) DG k 在场景 s 中的运行状态 (1= 可用, 0= 故障)

表 3: 符号说明 (续表)

符号	含义
$\text{RandFactor}_k(s)$	(问题 2) DG k 在场景 s 中的随机出力比例
E	(问题 1) 所有可能的故障事件集合
P_e	(问题 1) 故障事件 e 的发生概率
$\lambda_{\text{line}}, \lambda_{\text{switch}}, \lambda_{\text{DG/user}}, \lambda_{\text{unit}}$	线路、开关、DG/用户、单元的故障率
Load_j	(问题 1) 馈线 j 的负荷
L_i	(问题 1) 馈线 i 的本地负荷
D_j	(问题 2) 节点 j 的原始负荷需求
$D_j(s)$	(问题 2) 负荷 j 在场景 s 的随机波动值 (可选)
$L_{f'}$	(问题 2) 馈线 f' 的自身负荷
S_i	(问题 1) 馈线 i 的净出力盈余
$P_{\text{exportable},i}$	(问题 1) 馈线 i 可输出至相邻馈线的最大功率
$T_{\text{available}}^{(j)}$	(问题 1) 相邻馈线 j 的剩余容量
$S_{\text{tie-line to } j, \text{max}}$	(问题 1) 至馈线 j 的联络线最大传输容量
$S_{k, \text{max}}$	(问题 1) (从馈线 i 看) 至相邻馈线 k 的联络线最大传输容量
$T_{\text{actual}}^{(j)}$	(问题 1) 对馈线 j 的实际转供量
$T_{\text{available}}^{(f')}(s, \gamma)$	(问题 2) 健康馈线 f' 在场景 s 、 γ 下的可供转移容量
$P_{\text{feeder_cap}}^{(f')}$	(问题 2) 馈线 f' 的额定容量
$S_{t, \text{max}}$	(问题 2) 联络线 t 的最大传输容量
$T_{\text{actual}}^{(t)}(s, \gamma)$	(问题 2) 联络线 t 在场景 s 、 γ 下的实际转供量
P_k	(问题 1) 线路 k 的有功功率 (单位: MW)
I_k	(问题 1) 线路 k 的电流 (基于 P_k in MW)
I_{rated}	(问题 1) 馈线额定电流 (220A)
$P_{k,s}(\gamma)$	(问题 2) 线路 k 在场景 s 、 γ 下的有功功率 (单位: kW)
$I_{k,s}(\gamma)$	(问题 2) 线路 k 在场景 s 、 γ 下的电流 (基于 $P_{k,s}(\gamma)$ in kW)
$I_{k, \text{rated}}$	(问题 2) 线路 k 的额定电流 (220A)
$R_{\text{sys}}(\gamma)$	(问题 2) 容量因子 γ 下的系统总风险 (成本)

4 模型建立与求解

4.1 问题 1 模型建立与求解

4.1.1 问题 1 求解思路

4.1.2 问题 1 模型建立

为量化分布式能源接入配电网后的系统风险，本研究构建了基于概率建模与网络拓扑分析的综合评估框架。模型以系统风险公式 (1) 为核心，通过蒙特卡洛模拟生成故障场景，并结合动态功率分配与危害度函数计算，实现对失负荷与过负荷风险的系统性量化：

$$R_{\text{sys}} = P_{\text{LL}} C_{\text{LL}} + P_{\text{OL}} C_{\text{OL}}, \quad (1)$$

其中 R_{sys} 为系统总风险， P_{LL} 和 P_{OL} 分别为失负荷与过负荷的发生概率， C_{LL} 和 C_{OL} 分别为其对应的危害程度。

失负荷风险 R_{LL} 的计算公式为：

$$R_{\text{LL}} = \sum_{e \in E} P_e \cdot C_{\text{LL},e}, \quad (2)$$

其中 E 为所有可能的故障事件集合， P_e 为故障事件 e 的发生概率， $C_{\text{LL},e}$ 为事件 e 导致的失负荷危害度。

故障事件概率 P_e 通过单元故障率计算：线路故障率： $\lambda_{\text{line}} = \text{长度 (km)} \times 0.002/\text{km}$ ；开关故障率： $\lambda_{\text{switch}} = 0.2\%$ ；DG/用户故障率： $\lambda_{\text{DG/user}} = 0.5\%$ 。

对每个单元独立判断是否发生故障：若 $\text{Uniform}(0, 1) < \lambda_{\text{unit}}$ ，则标记为故障单元。例如，某线路长 5 km，其故障率为 $5 \times 0.002 = 1\%$ ，当随机数小于 0.01 时触发故障。

最终失负荷概率 P_{LL} 通过蒙特卡洛模拟统计失负荷事件频率获得：

$$P_{\text{LL}} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(L_{\text{LL},i} > 0), \quad (3)$$

其中指示函数 $\mathbb{I}(\cdot)$ 定义为：

$$\mathbb{I}(L_{\text{LL},i} > 0) = \begin{cases} 1, & \text{若场景 } i \text{ 发生失负荷} \\ 0, & \text{否则} \end{cases}. \quad (4)$$

接下来进行网络拓扑与转供能力分析。相邻馈线 j 的剩余容量 $T_{\text{available}}^{(j)}$ 计算如下：

$$T_{\text{available}}^{(j)} = \begin{cases} 2.2 \text{ MW} - (\text{Load}_j - \text{DG}_j), & \text{若 } \text{Load}_j \geq \text{DG}_j \\ 2.2 \text{ MW}, & \text{若 } \text{DG}_j > \text{Load}_j \end{cases}, \quad (5)$$

若联络开关故障（概率 0.2%），则 $T_{\text{available}}^{(j)} = 0$ 。

动态转供分配机制综合剩余容量、联络线物理上限及故障负荷需求，计算实际转供量：

$$T_{\text{actual}}^{(j)} = \min \left(T_{\text{available}}^{(j)}, S_{\text{tie-line to } j, \text{max}}, \frac{T_{\text{available}}^{(j)}}{\sum_k T_{\text{available}}^{(k)}} \cdot L_{\text{fault}} \right), \quad (6)$$

其中 $S_{\text{tie-line to } j, \max}$ 为联络线最大传输容量, L_{fault} 为故障导致的总失负荷量.

最终失负荷量 $L_{\text{LL},i}$ 为故障负荷与实际转供量的差值:

$$L_{\text{LL},i} = \max \left(L_{\text{fault}} - \sum_j T_{\text{actual}}^{(j)}, 0 \right), \quad (7)$$

若 $L_{\text{LL},i} > 0$, 则标记该场景为”失负荷发生”.

定义危害度函数 $F(L_j, j, t_j)$ 反映用户类型 j 、失负荷功率 L_j 及停电时长 t_j 的综合影响:

$$C_L(f_k) = \sum_{j \in L_{\text{final_lost}}(f_k)} F(L_j, j, t_j). \quad (8)$$

非线性客户停电损失函数 (CDFs) 形式为:

$$F(L_j, j, t_j) = L_j \cdot \left[a_j \cdot t_j^{b_j} + c_j \cdot \mathbb{I}(t_j > t_{\text{crit},j}) \right], \quad (9)$$

其中 a_j, b_j, c_j 为用户类型相关系数, $t_{\text{crit},j}$ 为关键停电时长阈值, $\mathbb{I}(\cdot)$ 为示性函数.

过负荷风险建模聚焦于分布式能源 (DG) 出力波动对馈线电流的影响, 并通过闭环优化实现经济性与物理安全性的协同控制. 模型采用双阶段策略: 前期以削减成本最小化为目标优化调度方案, 后期通过潮流计算验证物理可行性, 最终形成反馈机制以平衡两类风险.

经济性优化阶段通过量化 DG 出力冗余导致的本地削减成本, 优化功率分配策略. 模型输入数据包括: DG 出力概率分布、负荷需求、馈线额定容量 (2.2 MW) 及联络线传输限制、单位削减成本 $c_{\text{curtail},i}$ (如市场电价或补贴政策).

核心约束条件为: DG 不得向上级电网倒送功率, 但允许相邻馈线间调节功率. 具体步骤如下:

1. 计算馈线 i 的净出力盈余:

$$S_i = P_{\text{dg},i} - L_i,$$

其中 $P_{\text{dg},i}$ 为 DG 出力, L_i 为本地负荷.

2. 确定可输出至相邻馈线的最大功率:

$$P_{\text{exportable},i} = \sum_{k \in \mathcal{T}_i} \min(S_{k,\max}, 2.2 \text{ MW} - (\text{Load}_k - \text{DG}_k)),$$

其中 $\mathcal{T}_i$ 为馈线 i 的联络线集合, $S_{k,\max}$ 为相邻馈线 k 的剩余容量.

3. 判定需本地削减的功率:

$$P_{\text{curtail},i} = \max(0, S_i - P_{\text{exportable},i}).$$

4. 计算期望削减损失:

$$C_{\text{OL},i} = \mathbb{E}[P_{\text{curtail},i}] \cdot c_{\text{curtail},i}.$$

5. 最终目标函数为最小化系统过负荷经济风险:

$$R_{\text{OL}} = \sum_{i=1}^N C_{\text{OL},i}.$$

物理安全性验证阶段通过潮流计算验证优化方案是否满足电流约束，并反馈调整调度策略。模型输入包括优化后的 DG 出力、负荷分布及网络拓扑参数，关键假设为不考虑无功功率与电压越限影响。具体步骤如下：

1. 对每条线路 k ，计算电流 I_k 。基于 10 kV 馈线额定电流 $I_{\text{rated}} = 220 \text{ A}$ ，采用简化公式：

$$I_k = 100 \cdot P_k \quad (\text{当 } P_k \text{ 以 MW 为单位}),$$

其中 P_k 为线路有功功率。

2. 判断过载条件：若 $I_k > 1.1 \cdot I_{\text{rated}} = 242 \text{ A}$ ，则标记为过载线路。

3. 计算过载危害度：

$$C_{\text{OL}} = \sum_{k \in \mathcal{L}} c_k \cdot \left(\frac{I_k - 1.1 I_{\text{rated}}}{1.1 I_{\text{rated}}} \right)^2,$$

其中 c_k 为线路 k 的单位过载成本。该危害度函数采用超额电流的归一化平方形式，反映边际成本递增特性——电流越限越严重，设备损坏与稳定性损失呈非线性增长。

4. 统计过载概率 P_{OL} ：

$$P_{\text{OL}} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(I_k > 1.1 I_{\text{rated}}),$$

其中指示函数 $\mathbb{I}(\cdot)$ 定义为：

$$\mathbb{I}(I_k > 1.1 I_{\text{rated}}) = \begin{cases} 1, & \text{若线路 } k \text{ 过载} \\ 0, & \text{否则} \end{cases}.$$

5. 若 $P_{\text{OL}} > 5\%$ 或 C_{OL} 超过预设阈值，则触发反馈机制，调整 DG 出力限值或联络线功率分配规则。

该模型形成了闭环优化框架，模型通过迭代优化实现经济性与物理风险的平衡，流程如图 1 所示。

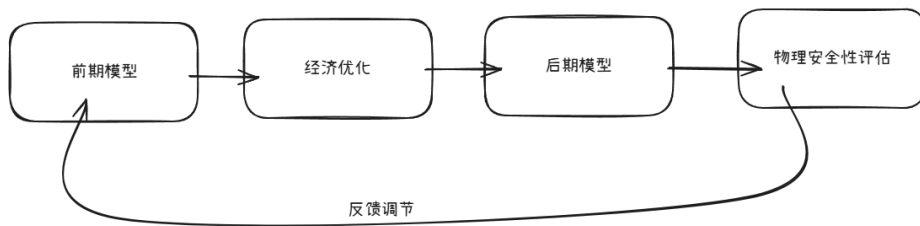


图 1: 闭环反馈流程图

介绍如下：

参数初始化： 设定 DG 出力上限 $P_{\text{dg},i} \leq L_i + P_{\text{exportable},i}$ ，预设联络线按剩余容量比例分配功率。

迭代优化： 前期模型计算削减成本，输出最优调度方案；后期模型验证物理风险，标记过载线路；调整参数（如降低 DG 出力限值、限制联络线传输功率），反馈调节。

收敛条件： 连续两次迭代中，经济损失与物理风险均低于预设阈值，且差异小于 1%。

4.1.3 问题 1 模型求解与分析

模型实现依托 Python 编程语言，利用 `networkx` 库进行拓扑分析，`pandas` 处理负荷与 DG 出力数据，并通过蒙特卡洛模拟生成故障场景. 动态转供分配机制与非线性危害度函数的设计提升了模型对复杂运行场景的适应性，但受限于故障率数据的准确性及大规模潮流计算的效率. 整体而言，该模型通过概率建模与网络约束的结合，科学地量化了失负荷与过负荷风险，为后续问题提供了坚实基础. 下图为可视化的节点网络建模，实现了按用户类型分类.

配电网拓扑结构与过负荷风险可视化（简化版）

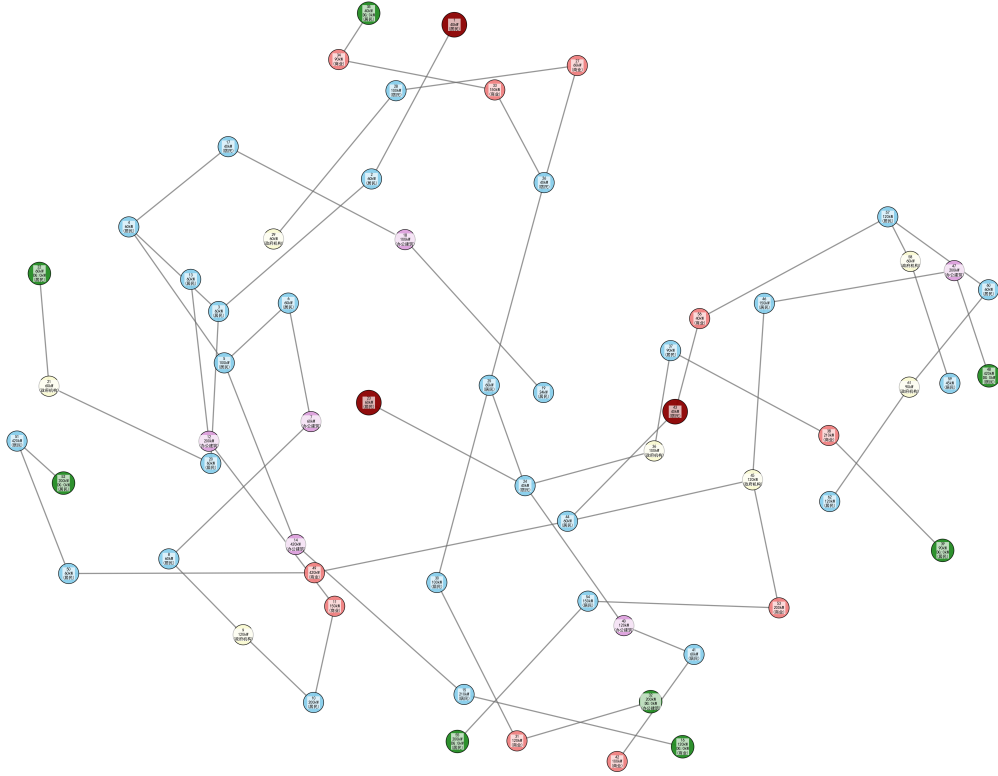


图 2: 节点网络可视化

4.2 问题 2 模型建立与求解

4.2.1 问题 2 求解思路

为分析在 62 节点有源配电网中，分布式能源（DG）不同接入容量对系统风险演变的影响，本研究构建了基于多场景分析的动态风险评估框架. 核心思路是，以 DG 的初始装机容量 I （例如，单个 DG 点 300kW）为基准，通过引入容量因子 γ ，系统地改变网络中所有 DG 的接入容量，进而评估系统总风险 R_{sys} 的变化情况.

具体求解步骤如下：

1. **参数序列定义与场景生成**：设定 DG 容量因子 γ 的变化范围为 $[1.0, 3.0]$ ，并以 0.3 为步长，形成一个梯度序列（即 $\gamma \in \{1.0, 1.3, 1.6, \dots, 3.0\}$ ）. 对于序列中的每一个 γ 值：

- 更新系统中所有 DG 的当前接入容量 $P_{dg,k}^{cap}(\gamma) = \gamma \cdot P_{dg,k}^{base}$ ，其中 $P_{dg,k}^{base}$ 为 DG 单元 k 的初始基准容量。
- 采用蒙特卡洛模拟方法生成 N_{sim} (10^4) 个系统运行场景。每个场景综合考虑以下不确定性因素：
 - **DG 出力波动**：DG 的实际出力 $P_{dg,k}^{out}(s, \gamma)$ 在其当前容量 $P_{dg,k}^{cap}(\gamma)$ 的一定百分比范围内随机波动。
 - **元件随机故障**：基于各元件（线路、开关、DG 单元、用户负荷节点）的年故障率，通过随机抽样判断其在当前场景 s 是否发生故障。线路故障率与其长度相关。
 - **负荷波动**：当前 Python 代码实现中使用的是 Excel 表中的确定性负荷，但模型框架可支持负荷 $D_j(s)$ 在其基准值附近随机波动。
- 2. **系统风险计算**：对于每个容量因子 γ 下的每一个运行场景 s ，利用问题 1 中建立的风险评估模型（DistributionNetworkRiskModel 类）计算该场景下的失负荷损失 $C_{LL,s}(\gamma)$ 和过负荷损失（包括 DG 削减成本和物理过载成本） $C_{OL,s}(\gamma)$ 。
- 3. **期望风险评估**：通过对 N_{sim} 个场景的损失结果进行平均，得到在该容量因子 γ 下的期望失负荷风险 $R_{LL}(\gamma)$ 和期望过负荷风险 $R_{OL}(\gamma)$ ，进而得到系统总风险 $R_{sys}(\gamma)$ 。
- 4. **风险演变分析**：将不同 γ 值对应的 $R_{sys}(\gamma)$ 、 $R_{LL}(\gamma)$ 和 $R_{OL}(\gamma)$ 进行可视化（绘制风险演变曲线），分析 DG 渗透率提升对系统各类风险的非线性影响规律，识别潜在的风险拐点和临界容量阈值。
- 5. **结果输出与报告**：将各容量因子下的详细风险数据保存，并生成分析报告，总结风险演变趋势及关键发现。

本求解思路通过系统的参数敏感性分析，量化了 DG 容量变化对配电网运行风险的驱动机制，为 DG 的优化规划和风险管理提供决策支持。技术实现上，依托 Python 语言及其科学计算库（Pandas, NumPy, NetworkX, Matplotlib）构建仿真分析平台。

4.2.2 问题 2 模型建立

为定量分析分布式能源（DG）不同接入容量对附录中 62 节点配电网系统风险 R_{sys} 的演变情况，本研究在问题 1 风险评估模型的基础上，进一步考虑 DG 容量的动态变化。模型的核心目标是揭示随着 DG 总容量从初始水平（I）增加至三倍（3I），系统失负荷风险与过负荷风险的变化规律。

系统总风险 R_{sys} 定义为失负荷风险 R_{LL} 与过负荷风险 R_{OL} 之和。引入 DG 容量因子 γ ，表示 DG 容量相对于其初始基准容量 $P_{dg,k}^{base}$ 的倍数。因此，特定 DG 单元 k 在容量因子为 γ 时的接入容量为 $P_{dg,k}^{cap}(\gamma) = \gamma \cdot P_{dg,k}^{base}$ 。系统风险是关于 γ 的函数：

$$R_{sys}(\gamma) = R_{LL}(\gamma) + R_{OL}(\gamma) \quad (10)$$

两种风险均通过蒙特卡洛模拟进行评估，考虑了电网元件的随机故障和 DG 出力的随机波动。

失负荷风险定义为在给定的 DG 容量因子 γ 下，因各类故障导致的期望停电损失成本。通过 N_{sim} 次蒙特卡洛模拟计算：

$$R_{\text{LL}}(\gamma) = \frac{1}{N_{\text{sim}}} \sum_{s=1}^{N_{\text{sim}}} C_{\text{LL},s}(\gamma) \quad (11)$$

其中， $C_{\text{LL},s}(\gamma)$ 为第 s 次模拟场景下，当 DG 容量因子为 γ 时，系统发生的总失负荷危害度。其计算方式如下：

$$C_{\text{LL},s}(\gamma) = \sum_{j \in \mathcal{N}_{\text{unserved}}^s} F(L_{j,s}^{\text{unserved}}(\gamma), \text{type}_j, t_j)$$

其中 $L_{j,s}^{\text{unserved}}(\gamma)$ 是节点 j 在场景 s 下的最终未服务负荷， type_j 是用户类型， t_j 是预设的停电持续时间。客户危害度函数 $F(\cdot)$ 的具体形式（线性或非线性）已在问题 1 模型中定义。

$L_{j,s}^{\text{unserved}}(\gamma)$ 的确定过程如下：

DG 实际出力：在场景 s 中，DG 单元 k 的实际出力 $P_{\text{dg},k}^{\text{out}}(s, \gamma)$ 取决于其当前容量 $P_{\text{dg},k}^{\text{cap}}(\gamma)$ 、自身的运行状态（是否故障）以及一个随机出力系数（例如，在其当前容量的 50%-100% 之间随机波动）。

$$P_{\text{dg},k}^{\text{out}}(s, \gamma) = \text{OpStatus}_k(s) \cdot \text{RandFactor}_k(s) \cdot P_{\text{dg},k}^{\text{cap}}(\gamma) \quad (12)$$

其中 $\text{OpStatus}_k(s)$ 为 1 表示 DG k 在场景 s 中可用，为 0 表示故障； $\text{RandFactor}_k(s)$ 为随机出力比例。

故障模拟：模拟线路、开关（包括联络开关和变电站出线开关）、DG 单元及负荷点（用户侧）的随机故障，生成故障后的网络拓扑 G_s 。

孤岛判定与初始失负荷：分析 G_s ，找出所有与有效电源点（未故障的变电站出线开关所连接的节点）失去连接的孤岛 $\mathcal{I}_s$ 。对于每个孤岛，其初始未满足负荷为：

$$L_{\text{island_initial_unserved},s}(\gamma) = \max \left(0, \sum_{j \in \mathcal{I}_s, j \notin \mathcal{F}_s} D_j - \sum_{k \in \text{DG in } \mathcal{I}_s} P_{\text{dg},k}^{\text{out}}(s, \gamma) \right) \quad (13)$$

其中 D_j 是节点 j 的原始负荷需求， $\mathcal{F}_s$ 表示故障负荷集合。

联络线负荷转供：

对于每个能够向孤岛 $\mathcal{I}_s$ 供电的健康馈线 f' （通过未故障的联络开关 t 连接），计算其可供转移容量 $T_{\text{available}}^{(f')}(s, \gamma)$ 。这取决于馈线 f' 的额定容量 $P_{\text{feeder_cap}}^{(f')}$ 及其当前的净负荷（考虑其自身负荷和内部 DG 的实际出力 $P_{\text{dg},f'}^{\text{out}}(s, \gamma)$ ，且 DG 不倒送功率）：

$$T_{\text{available}}^{(f')}(s, \gamma) = \max \left(0, P_{\text{feeder_cap}}^{(f')} - \max(0, L_{f'} - P_{\text{dg},f'}^{\text{out}}(s, \gamma)) \right) \quad (14)$$

联络线 t 的最大传输容量为 $S_{t,\text{max}}$ 。

若有多条路径可向孤岛 $\mathcal{I}_s$ 转供，其实际转供量 $T_{\text{actual}}^{(t)}(s, \gamma)$ 根据各路径的 $\min(T_{\text{available}}^{(f')}(s, \gamma), S_{t,\text{max}})$ 和孤岛的总未满足负荷 $L_{\text{island_initial_unserved},s}(\gamma)$ 按比例动态分配：

$$T_{\text{actual}}^{(t)}(s, \gamma) = \min \left(T_{\text{available}}^{(f')}(s, \gamma), S_{t,\text{max}}, \frac{\min(T_{\text{available}}^{(f')}(s, \gamma), S_{t,\text{max}})}{\sum_{t' \in \mathcal{P}_{\text{island}}} \min(T_{\text{available}}^{(f'')}(s, \gamma), S_{t',\text{max}})} \cdot L_{\text{island_initial_unserved},s}(\gamma) \right) \quad (15)$$

孤岛的最终未满足负荷 $L_{\text{island_final_unserved},s}(\gamma) = \max(0, L_{\text{island_initial_unserved},s}(\gamma) - \sum_t T_{\text{actual}}^{(t)}(s, \gamma))$. 此负荷随后按比例分配到孤岛内的各个用户节点 j , 得到 $L_{j,s}^{\text{unserved}}(\gamma)$.

过负荷风险定义为在给定 DG 容量因子 γ 下, 因 DG 出力波动和网络潮流分布导致的期望物理过载损失成本. DG 出力不进行主动削减优化, 直接评估其随机出力下的物理风险.

$$R_{OL}(\gamma) = \frac{1}{N_{\text{sim}}} \sum_{s=1}^{N_{\text{sim}}} C_{OL,s}(\gamma) \quad (16)$$

其中, $C_{OL,s}(\gamma)$ 为第 s 次模拟场景下的总物理过载危害度. 计算步骤如下:

DG 实际出力: 同失负荷风险模型中的 $P_{dg,k}^{\text{out}}(s, \gamma)$.

线路功率估算: 由于配电网通常具有辐射状或弱环网结构, 本模型对每个馈线内部的潮流进行简化估算. 假设馈线由其变电站出口点 (如 CB1, CB2, CB3 对应的节点) 供电, 且内部为辐射状. 通过深度优先搜索 (DFS) 确定从根节点到各节点的路径, 线路 k (连接父节点 u 和子节点 v) 的有功功率 $P_{k,s}(\gamma)$ 估算为以子节点 v 为根的下游所有节点的净负荷之和 (节点负荷 - 节点 DG 实际出力).

线路电流计算: 线路 k 的电流 $I_{k,s}(\gamma)$ 按下式计算:

$$I_{k,s}(\gamma) = 100 \cdot P_{k,s}(\gamma) \quad (17)$$

其中 $P_{k,s}(\gamma)$ 单位为 MW.

过载判断与危害度计算: 线路额定电流为 $I_{k,\text{rated}} = 220$ A. 若 $I_{k,s}(\gamma) > 1.1 \cdot I_{k,\text{rated}}$, 则线路 k 发生过载. 其危害度 $C_{OL,k,s}(\gamma)$ 计算如下:

$$C_{OL,k,s}(\gamma) = c_k \cdot \left(\frac{I_{k,s}(\gamma) - 1.1 \cdot I_{k,\text{rated}}}{1.1 \cdot I_{k,\text{rated}}} \right)^2 \quad (18)$$

其中 c_k 为线路 k 的单位过载成本系数. 场景总物理过载危害度为 $C_{OL,s}(\gamma) = \sum_{k \in \mathcal{O}_s} C_{OL,k,s}(\gamma)$.

系统过载概率: 作为辅助指标, 统计在 N_{sim} 次模拟中, 至少有一条线路发生过载的场景比例 $P_{OL,\text{sys}}(\gamma)$:

$$P_{OL,\text{sys}}(\gamma) = \frac{1}{N_{\text{sim}}} \sum_{s=1}^{N_{\text{sim}}} \mathbb{I}(\exists \text{线路 } k, s.t. I_{k,s}(\gamma) > 1.1 \cdot I_{k,\text{rated}}) \quad (19)$$

通过在 1.0 到 3.0 范围内以 0.3 为步长调整 DG 容量因子 γ , 运行上述蒙特卡洛模拟, 得到不同 γ 值下的 $R_{LL}(\gamma)$ 和 $R_{OL}(\gamma)$, 进而得到系统总风险 $R_{\text{sys}}(\gamma)$ 的演变曲线. 这将揭示 DG 容量增长对配电网各类风险的具体影响模式.

本模型实现依托 Python 及其科学计算库, 其中 `networkx` 用于处理网络拓扑分析, `numpy` 和 `pandas` 用于数据处理和计算, `matplotlib` 用于结果可视化. 线路功率估算采用了基于辐射状假设的简化方法, 对于高度环网的系统, 其精度可能受限, 采用更专业的潮流计算工具 (如 `Pandapower`) 可进一步提升准确性.

4.2.3 问题 2 模型求解与分析

下表展示了不同 DG 容量因子下, 系统总 DG 容量、失负荷风险成本、过负荷风险成本以及系统总风险成本的模拟结果.

表 4: 不同 DG 容量因子下的风险演变结果

DG 容量因子 (γ)	总 DG 容量 (kW)	失负荷风险成本 (R_{LL})	过负荷风险成本 (R_{OL})	系统总风险成本 (R_{sys})
1.0	2400.0	81.489525	0.314721	81.804246
1.3	3120.0	88.773237	0.114648	88.887886
1.6	3840.0	84.735198	0.027151	84.762349
1.9	4560.0	89.830245	0.003864	89.834109
2.2	5280.0	69.944509	0.000211	69.944720
2.5	6000.0	68.444804	0.000000	68.444804
2.8	6720.0	51.050645	0.000000	51.050645
3.0	7200.0	33.015026	0.000000	33.015026

从表 4 可以看出, 过负荷风险成本随着 DG 容量因子的增加而急剧下降. 当 DG 容量因子为 1.0 时, 过负荷风险成本约为 0.31. 随着因子增加到 1.3, 风险降至 0.11 左右. 当容量因子达到 2.5 及以上时, 过负荷风险成本已降至 0.00, 表明在此模拟条件下, 较高的 DG 渗透率显著降低了由于线路潮流超限或 DG 出力无法消纳 (如果模型包含削减成本) 所引发的过负荷风险. 这可能是因为 DG 的增加满足了更多的本地负荷, 减少了主干线路的传输压力, 或者在模型中有效地通过联络线等方式消纳了多余电力. 失负荷风险成本的演变趋势较为复杂. 初始时 ($\gamma = 1.0$), 风险成本为 81.49. 当容量因子增加到 1.3 和 1.9 时, 失负荷风险反而略有上升, 分别达到 88.77 和 89.83. 这可能反映了在 DG 容量中等增加但尚未达到显著支撑水平时, DG 出力的随机性或故障可能对局部供电可靠性带来一定的负面扰动, 或者模拟的随机故障场景恰好在这些容量水平下导致了较高的失负荷. 然而, 当 DG 容量因子进一步增加到 2.2 及以上时, 失负荷风险成本开始显著且持续下降. 在 $\gamma = 2.2$ 时降至 69.94, 在 $\gamma = 2.5$ 时为 68.44, 在 $\gamma = 2.8$ 时大幅降至 51.05, 并在 $\gamma = 3.0$ 时达到最低点 33.02. 这表明, 当 DG 容量足够大时, 其对本地负荷的支撑能力、形成孤岛运行的能力以及通过联络线支援故障区域的能力得到显著增强, 从而有效降低了因元件故障导致的失负荷. 系统总风险成本主要由失负荷风险主导, 因为过负荷风险的量级相对小得多. 因此, R_{sys} 的演变趋势与 R_{LL} 非常相似. 系统总风险在 DG 容量因子为 1.9 时达到一个局部高点 (89.83), 之后随着 DG 容量的进一步增加而稳步下降.

根据模拟结果, 可以得出以下主要结论:

- DG 容量对过负荷风险的显著改善作用:** 在本模型和参数设定下, 增加 DG 容量能非常有效地降低系统的过负荷风险, 当 DG 容量因子达到 2.5 及以上时, 过负荷风险几乎可以忽略不计.
- DG 容量对失负荷风险的非单调影响与最终改善:** 失负荷风险并非随 DG 容量的增加而单调下降. 在 DG 容量中等增加的阶段, 风险可能存在小幅波动甚至上升. 但当 DG 容量达到较高水平 (本例中 $\gamma \geq 2.2$) 后, 其对供电可靠性的正面效应开始凸显, 失负荷风险显著降低.
- 最优 DG 容量因子:** 在所考察的 DG 容量因子范围内 (1.0 至 3.0), 系统总风险成本在

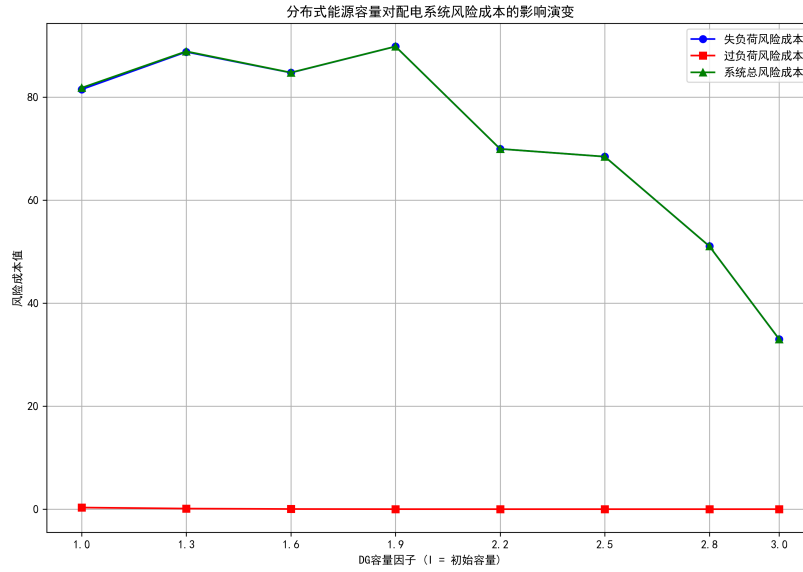


图 3: DG 容量对系统风险的影响曲线

DG 容量因子 $\gamma = 3.0$ 时达到最低, 为 33.02. 此时, 总 DG 容量为 7200 kW. 在该最优状态下, 系统风险几乎完全由失负荷风险构成 (33.02), 过负荷风险已降至 0.

基于以上分析, 建议在配电网规划和 DG 接入评估时:

- 考虑 DG 接入对两类风险的综合影响, 而非仅关注其对某一类风险的单方面作用.
- 认识到 DG 容量的增加并非总能立即带来失负荷风险的降低, 可能存在一个“阵痛期”或平台期. 需要达到一定的渗透率和总容量后, 其对可靠性的提升作用才更为明确.
- 在本案例的特定条件下, 进一步增加 DG 容量 (在 3I 的水平上) 似乎仍能带来系统总风险的降低. 然而, 实际决策中还需要考虑 DG 接入的投资成本、对电能质量的潜在影响以及更极端故障场景下的系统表现, 进行更全面的成本效益分析.
- 针对失负荷风险在中等 DG 容量下可能存在的波动, 应研究相应的协调控制策略, 以平滑 DG 出力波动或优化故障恢复过程.

未来的研究可以进一步考虑不同类型 DG 的出力特性、储能系统的配置以及更精细化的网络运行约束, 以获得更全面的风险评估结果.

5 模型分析检验

为验证所构建配电网风险评估模型的稳健性与可靠性, 本节对模型的关键参数进行灵敏度分析, 并探讨了模型可能存在的误差来源及控制方法.

5.1 灵敏度分析

灵敏度分析旨在评估模型输出结果对输入参数变化的敏感程度, 从而识别对系统风险影响显著的关键因素. 本研究选取了三个核心参数进行分析: 线路单位长度故障率 (λ_{line})、DG 削减成本 (c_{curtail}) 以及线路过载成本系数 (c_k). 在进行某一参数的灵敏度分析时, 保持其他参数为基准值, 并固定 DG 容量因子为 1.0 (即各 DG 点容量为初始的 300kW) .

分析结果表明, 线路单位长度故障率对系统总风险的影响最为显著. 当线路故障率从 0.001/km 增加至 0.005/km 时, 系统总风险成本的均值从约 60.36 上升至约 210.82, 标准差也相应增大, 显示出失负荷风险是该参数变化下的主要驱动因素. 具体而言, 失负荷风险成本均值从约 60.04 大幅增加到约 210.51, 而过负荷风险成本均值在此参数变化范围内保持在 0.31 左右, 波动极小. 这说明在当前模型设定和 DG 渗透率水平下, 线路的可靠性是影响系统失负荷损失的关键. DG

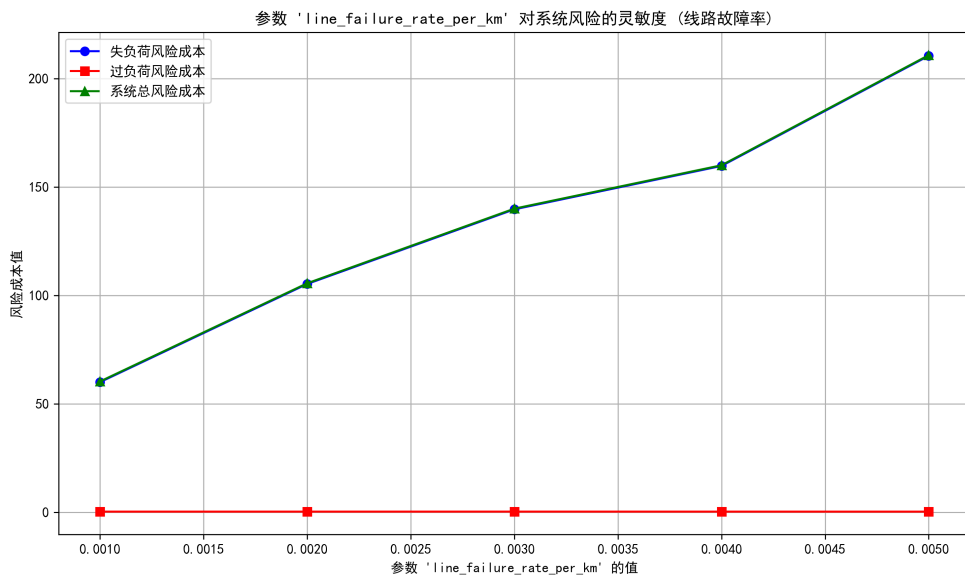


图 4: 线路故障率对系统风险的影响曲线图

削减成本对系统总风险的影响也较为明显, 但主要体现在过负荷风险中的经济削减成本部分. 当 DG 削减成本参数从 0.1 变化至 0.5 时, 系统总风险成本均值从约 61.36 变化至约 85.93. 失负荷风险成本在此过程中也表现出一定的波动, 均值从 61.04 变化至 85.61, 这可能间接反映了当削减成本变化时, 模型在优化 DG 出力与接纳负荷之间的平衡策略可能发生改变, 从而影响到故障场景下的负荷满足情况. 然而, 过负荷风险成本 (如此处主要指物理过载部分) 的均值仍维持在 0.315 附近, 标准差也较小, 说明 DG 削减成本的变化对物理线路过载的直接影响不显著.

线路过载成本系数 c_k 的变化对系统总风险的影响相对前两者较小, 但直接作用于过负荷风险的量级. 当该系数从 0.5 变化至 2.5 时, 系统总风险成本均值从约 82.08 上升至约 99.45. 其中, 过负荷风险成本均值从 0.17 线性增长至 0.77 左右, 失负荷风险成本均值则在 91.77 附近小幅波动. 这符合预期, 即物理过载的惩罚权重直接决定了该部分风险在总风险中的占比.

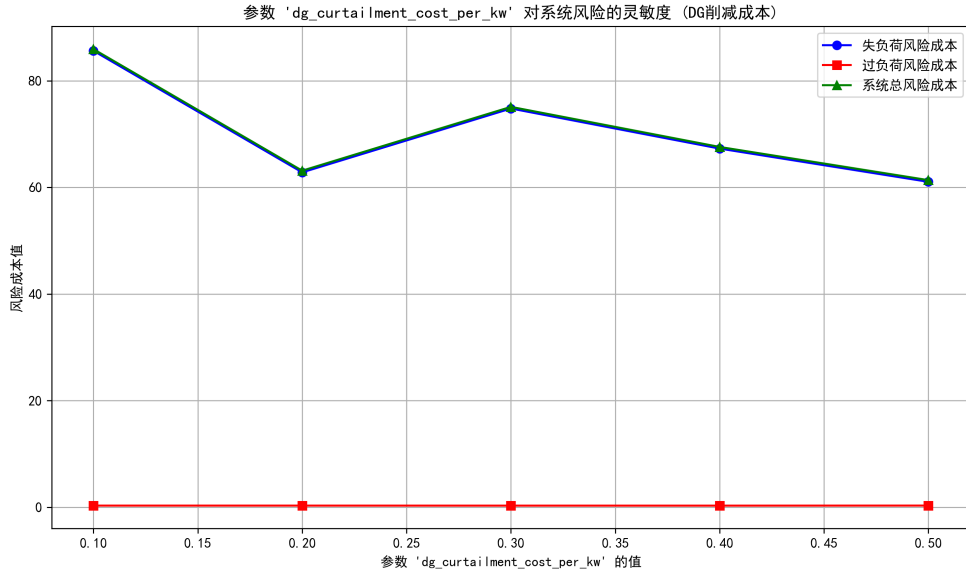


图 5: DG 削减成本对系统风险影响曲线图

综上所述, 系统对线路故障率最为敏感, 其次是 DG 削减成本, 而线路过载成本系数则直接调节物理过载风险的量级. 在进行风险管理和电网规划时, 应优先关注和提升线路的运行可靠性.

5.2 误差分析

模型的误差来源主要包括输入数据的不确定性、模型结构的简化以及蒙特卡洛模拟的收敛性.

输入数据方面, 各类故障率 (线路、开关、DG、用户)、DG 出力分布参数、负荷预测值以及经济参数 (如停电损失成本、削减成本、过载成本系数) 均是基于统计或预测得到, 本身存在一定的不确定性. 这些参数的微小变动可能通过模型的非线性传递而被放大, 导致最终风险评估结果的偏差. 例如, 灵敏度分析已显示故障率的变动对系统风险有显著影响.

模型结构简化是另一个主要的误差来源. 当前模型对线路潮流的估算采用了基于馈线辐射状运行的假设, 通过深度优先搜索计算下游负荷的汇集来近似线路功率. 这种方法未进行精确的交流潮流计算, 忽略了环网运行的可能性、无功功率的影响以及节点电压的精确分布, 可能导致线路功率和电流的估算不够准确, 尤其是在 DG 高渗透率或网络结构复杂的情况下. 此外, DG 出力模型 (例如, 从 Beta 分布简化为均匀分布)、元件故障模型 (如未考虑威布尔寿命分布的精确参数) 以及联络线动态转供策略的简化, 都可能引入模型误差.

蒙特卡洛模拟的收敛性直接关系到评估结果的稳定性和可靠性. 为检验模拟次数对结果稳定性的影响, 本研究固定其他参数 (DG 容量因子为 1.0), 分析了不同模拟次数 (从 100 次至 5000 次) 下系统总风险成本的均值和标准差. 结果显示, 当模拟次数从 100 增加到 500 时, 系统总风险的均值有一定波动, 标准差也相对较大. 随着模拟次数进一步增加到 1000 次、2000 次, 风险均值逐渐稳定, 标准差显著减小. 当模拟次数达到 5000 次时, 三次独立重复运行得到的系统总风险均值非常接近 (例如, 均值约为 70.02, 标准差降至 0.86 左右), 表明结果已具有较好的收敛性. 因此, 为确保风险评估结果的可靠性, 选择足够大的模拟次数至关重要. 在问题 2 的

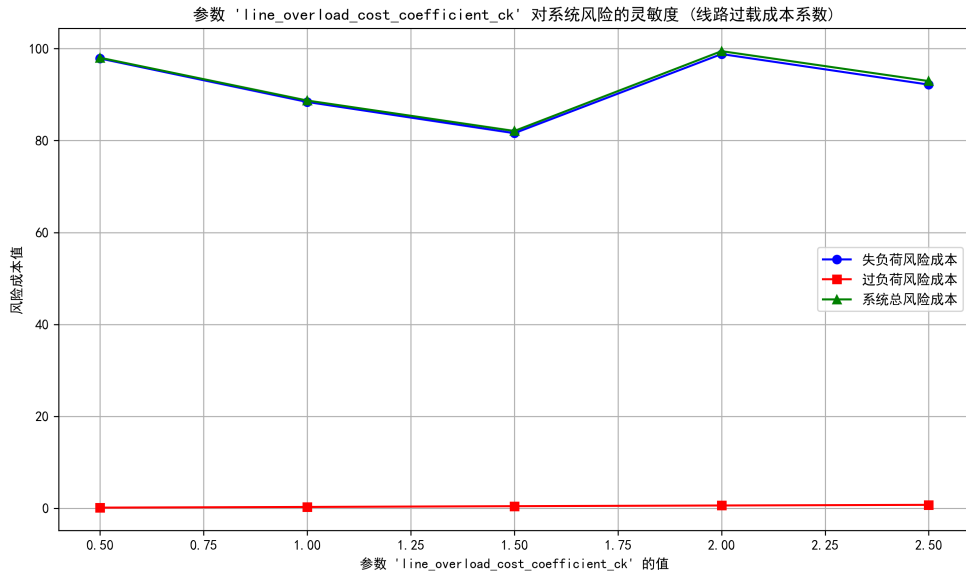


图 6: 线路过载成本系数 c_k 对系统风险的影响曲线图

DG 容量演变分析中, 采用了 $N_{\text{sim}} = 10^4$ 的模拟次数, 以期获得更为稳健的风险评估.

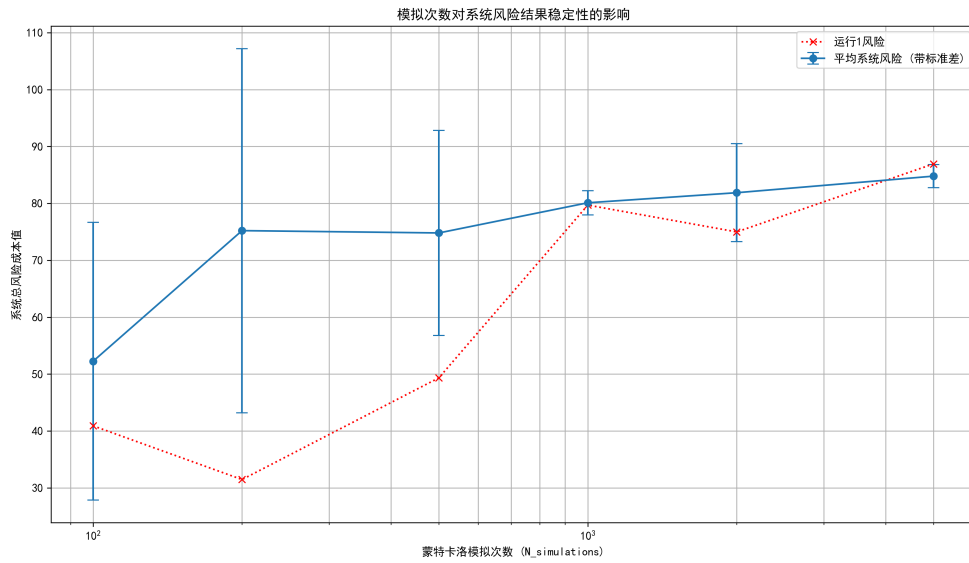


图 7: 不同模拟次数下计算系统风险稳定性

综上, 虽然模型通过蒙特卡洛模拟和参数分析提供了一套有效的风险评估框架, 但在解读结果时应充分考虑上述误差来源. 未来的工作可通过引入更精细的物理模型 (如精确潮流计算)、更准确的参数估计以及对输入数据不确定性的概率建模 (如区间分析或概率潮流) 来进一步提升模型的精度和鲁棒性.

6 模型评价与推广

本研究构建的配电网风险评估模型，通过蒙特卡洛模拟综合考虑了分布式能源（DG）接入带来的失负荷风险与过负荷风险。模型在参数化、不确定性处理以及风险因素的量化方面进行了一系列探索。

6.1 模型的优点

所建立的风险评估模型具备以下主要优点：首先，模型构建了一个较为全面的风险评估框架，能够同时量化由于元件故障导致的失负荷风险，以及由于 DG 出力波动和网络约束引发的过负荷风险。特别地，过负荷风险细分为 DG 经济削减成本和线路物理过载成本两部分，使得风险构成更为清晰。其次，模型通过蒙特卡洛模拟有效处理了系统运行中的关键不确定性，包括 DG 的随机出力（在其容量的一定百分比范围内波动）和各类元件（线路、开关、DG 单元、负荷节点）的随机故障，增强了风险评估结果的现实意义。再次，模型具有较好的参数化和灵活性。诸如元件故障率、DG 容量、各类成本系数（停电损失、DG 削减、线路过载）、联络线容量等关键参数均可调整，便于进行不同场景下的风险评估和参数敏感性分析，如问题 2 中对 DG 容量变化的风险演变分析。此外，模型在细节层面考虑了配电网的若干重要运行特性。在失负荷风险评估中，包含了对故障后孤岛的判定、孤岛内 DG 的支撑作用以及通过联络线进行负荷转供的模拟，并可区分不同用户类型的停电损失（通过线性或非线性客户危害度函数 CDF）。在过负荷风险评估中，遵循了 DG 不允许向主网倒送功率的基本原则，并模拟了因馈线间功率无法完全消纳而产生的 DG 经济削减。最后，该模型基于 Python 及常用的科学计算库（Pandas, NumPy, NetworkX, Matplotlib）构建，具有较好的模块化基础和代码可读性，为后续的功能扩展和算法改进提供了便利，并已集成了初步的可视化（网络拓扑、风险曲线）和报告生成功能。

6.2 模型的不足

尽管本模型在风险评估方面提供了一个综合框架，但仍存在一些不足之处有待进一步完善：其一，最为显著的不足在于线路潮流计算的简化。当前模型在评估线路功率和电流时，主要依赖于对馈线内部辐射状结构的假设，并通过深度优先搜索（DFS）估算下游负荷的汇集。此方法未能进行精确的交流潮流计算，因此无法准确反映网络内部各支路的真实潮流分布和节点电压水平，也未考虑无功功率对系统的影响。这直接限制了物理过载风险和潜在电压越限风险评估的精度，尤其对于包含环网运行或网孔结构的配电网，该简化可能导致较大误差。其二，DG 出力和负荷模型相对简化。DG 的随机出力当前采用较为基础的均匀分布（例如，容量的 50%-100%），未能采用更符合光伏、风电等实际出力特性的概率分布模型（如 Beta 分布、Weibull 分布），也未考虑天气因素、DG 间的出力相关性以及出力的时序特性。类似地，负荷模型当前主要基于静态的确定性数据，未引入负荷的随机波动或详细的时序变化特性。其三，元件故障模型可以进一步深化。目前故障率多为定值，未充分考虑元件老化状态、运行环境、历史故障数据等因素对故障概率的动态影响（例如，采用威布尔寿命分布模型进行更精细的故障建模）。同时，模型未区分故障的具体模式（如单相接地、相间短路等）及其不同的影响范围和修复时间分布。其四，经济性优化与物理安全性验证的耦合机制尚不完善。虽然论文思路中提及了双阶段优化和闭环反馈，但当前过负荷风险计算代码主要是先进行一步 DG 削减（经济性考量），然后基于削减后的出力

进行物理过载验证，尚未形成一个动态调整 DG 出力上限或联络线策略的迭代优化闭环。最后，模型的计算效率对于大规模网络和极高次数的蒙特卡洛模拟可能面临挑战。虽然 Python 生态系统提供了优化工具，但当前实现未充分利用并行计算（如 multiprocessing）或代码加速技术（如 Numba）来提升大规模仿真的性能。

6.3 模型的推广

基于当前模型的基础和已识别的不足，未来可以从以下几个方面进行模型的推广和深化，以增强其准确性、功能性和实用性：

1. **集成精确潮流计算模块：**将当前简化的潮流估算替换为基于牛顿-拉夫逊法或其他标准算法的精确交流潮流计算模块（例如，通过集成 Pandapower 等专业电力系统分析库）。这将显著提升线路负载、节点电压和无功潮流评估的准确性，从而更可靠地评估物理过载风险、电压越限风险，并为更复杂的控制策略提供基础。
2. **完善不确定性来源建模：**
 - **DG 出力模型：**引入基于气象数据（光照强度、风速）的 DG 出力模型，采用 Beta 分布（光伏）、Weibull 分布（风电）等更精确的概率分布，并考虑 DG 出力的时空相关性（例如，利用 Copula 函数）。
 - **负荷模型：**引入负荷的时序特性（如典型日负荷曲线）和随机波动性（如正态分布叠加在时序曲线上），并考虑负荷与 DG 出力的相关性。
3. **深化元件故障与恢复模型：**
 - 采用更精细的元件故障率模型，如考虑环境因素、运行年限的威布尔分布。
 - 区分不同的故障类型及其影响。
 - 引入随机的故障持续时间和修复时间模型，以更准确地评估停电损失。
4. **动态风险评估与时序模拟：**结合典型运行场景（如高峰、低谷、季节性变化、极端天气事件）构建时序场景库，通过马尔可夫链等方法建模场景间的状态转移过程，开展时序蒙特卡洛模拟，捕捉风险的动态演化特征，建立时空耦合的风险评估体系。
5. **优化网络重构与运行策略：**
 - 研究基于风险最小化的网络动态重构策略，例如，在故障发生后，通过优化联络开关和分段开关的状态，形成最优的供电恢复路径。
 - 实现论文思路中提及的经济性优化与物理安全性验证的闭环反馈机制，通过迭代调整 DG 出力上限、储能充放电策略或联络线功率分配规则，实现“安全-经济”双目标的协同优化。
6. **扩展风险评估维度：**
 - 引入电压越限风险、系统暂态稳定风险等其他关键运行风险的评估模块。

- 结合节点重要度指标（如电气介数、PageRank）、潮流重定向系数等进行网络拓扑脆弱性分析，识别关键节点和线路。
 - 利用 GIS 数据融合，创建风险热力图，实现系统脆弱区域的可视化定位与预警。
7. **提升计算效率与实用性：**针对大规模网络和高精度模拟需求，采用并行计算（如 ‘multiprocessing’ 库）、代码编译优化（如 ‘Numba’ 库）等技术提升仿真效率。开发用户友好的参数输入接口和结果展示界面，增强模型的工程实用性。
 8. **模型验证与实际应用：**将模型应用于实际配电网案例，利用历史运行数据和故障数据对模型参数进行校准和验证，评估模型的实际预测能力和决策支持效果。

通过上述推广，可以构建一个更为全面、精确和实用的配电网风险评估与优化决策平台，为高比例 DG 接入下配电网的安全、经济、可靠运行提供有力支撑。

参考文献

- [1] Zhang, Y.-M Zhang, Z.-H Yao, F. He, L.-Z Xu, W.. (2013). Risk assessment of power system components based on the risk theory. *Dianli Xitong Baohu yu Kongzhi/Power System Protection and Control*. 41. 73-78.
- [2] HART D G, AGÜERO J R, HUNT R, et al. K.D. JONES, L. HINTOS Dominion Energy USA[J].
- [3] Azmy, Ahmed and Erlich, I.. Impact of distributed generation on the stability of electrical power system. 2005 IEEE Power Engineering Society General Meeting, 2005. pp. 1056 - 1063 Vol. 2. DOI: 10.1109/PES.2005.1489354.
- [4] Majeed, Issah Nwulu, Nnamdi. (2022). Impact of Reverse Power Flow on Distributed Transformers in a Solar-Photovoltaic-Integrated Low-Voltage Network. *Energies*. 15. 9238. 10.3390/en15239238.
- [5] 李伟, 王刚, 张静. (2020). 配电网风险评估中的概率建模方法. *电力系统自动化*, 44(6), 1-9.
- [6] Majeed, I.B.; Nwulu, N.I. Impact of Reverse Power Flow on Distributed Transformers in a Solar-Photovoltaic-Integrated Low-Voltage Network. *Energies* 2022, 15, 9238. <https://doi.org/10.3390/en15239238>

A 附录

模型文件使用方法

先在总体文件夹下创建并激活虚拟环境

```
python3 -m venv myenv
```

```
source myenv/bin/activate
```

再安装依赖 `pip install -r requirements.txt` 然后在虚拟环境下运行即可。

Q1.py 为第一小问的代码（运行小于 1min）

Q2.py 为第二小问的代码（运行约 2min）

Q_sensitivity_error_analysis.py 为第三小问的代码（运行时间约 1min）

problem1_results_risk_calc 为第一题结果

问题 2 分析结果为第二题结果

灵敏度与误差分析结果为第三题结果

A 题附件的内容与位置是必要的，因为动过了 excel 表格内容

问题（1）代码

```
1 import numpy as np
2 import pandas as pd
3 import networkx as nx
4 import matplotlib.pyplot as plt
5 import os
6 import random
7 import matplotlib.colors as mcolors
8 # 中文显示设置
9 # 请确保您的环境已安装 SimHei 字体，或者替换为您可用的中文字体
10
11 plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
12 plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
13
14
15
16 def load_data_from_excel(excel_path):
17     """
18     从指定的Excel文件加载节点和线路数据。
19
20     参数:
21     excel_path (str): Excel文件的路径。
22
23     返回:
24     tuple: (nodes_df, lines_df)
25             nodes_df (pd.DataFrame): 包含节点负荷信息的DataFrame。
26                                     预期列: '有功P/kW' (从 "有功P/kW" 读
```

取, "No."列被忽略, 顺序决定节点ID)。

```
lines_df (pd.DataFrame): 包含线路拓扑信息的DataFrame。  
预期列: '编号', '起点', '终点', '长度/  
km', '电阻/ $\Omega$ ', '电抗/ $\Omega$ '。
```

```
"""
```

```
# 读取表1 (节点数据)
```

```
nodes_df = pd.read_excel(excel_path, sheet_name='表1', usecols=["有功P/  
kW"])
```

```
# 读取表2 (线路数据)
```

```
lines_df = pd.read_excel(excel_path, sheet_name='表2')  
expected_line_cols = ['编号', '起点', '终点', '长度/km', '电阻/ $\Omega$ ', '电  
抗/ $\Omega$ ']
```

```
print(f"成功从 '{excel_path}' 加载数据。")
```

```
return nodes_df, lines_df
```

```
class DistributionNetworkRiskModel:
```

```
def __init__(self, nodes_data, lines_data, n_simulations=1000):
```

```
    """
```

```
    初始化配电网风险模型
```

```
    参数:
```

```
nodes_data (pd.DataFrame): 包含节点负荷信息的DataFrame, 列: '有功P/  
kW'
```

```
lines_data (pd.DataFrame): 包含线路拓扑信息的DataFrame, 列: '起点',  
'终点', '长度/km', '电阻/ $\Omega$ ', '电抗/ $\Omega$ '
```

```
    """
```

```
self.nodes_data = nodes_data
```

```
self.lines_data = lines_data
```

```
self.n_simulations = n_simulations
```

```
# 记录节点的负荷功率
```

```
self.node_loads = {}
```

```
for i, row in nodes_data.iterrows():
```

```
    self.node_loads[i + 1] = row['有功P/kW']
```

```
# 定义节点分类
```

```
self.node_categories_map = {}
```

```

63     residential_nodes = [1, 2, 3, 4, 5, 6, 8, 10, 13, 15, 17, 19, 20,
64     22, 23, 24, 25, 26, 28, 30, 35, 37, 39, 41,
65     43, 44, 46, 48, 50, 51, 52, 54, 55, 57, 59,
66     60, 62]
67     commercial_nodes = [11, 16, 27, 31, 33, 34, 38, 42, 49, 53, 56]
68     government_nodes = [9, 21, 29, 36, 45, 58, 61] # 政府和机构
69     office_nodes = [7, 12, 14, 18, 32, 40, 47] # 办公和建筑
70
71     for node_id in range(1, len(self.nodes_data) + 1): # 遍历所有可能
72     的节点ID
73         if node_id in residential_nodes:
74             self.node_categories_map[node_id] = "居民"
75         elif node_id in commercial_nodes:
76             self.node_categories_map[node_id] = "商业"
77         elif node_id in government_nodes:
78             self.node_categories_map[node_id] = "政府机构"
79         elif node_id in office_nodes:
80             self.node_categories_map[node_id] = "办公建筑"
81         else:
82             self.node_categories_map[node_id] = "未知类别" # 为未分类
83     的节点指定默认类别
84
85     # 记录DG的位置和容量
86     self.dg_data = {}
87
88     # 系统参数
89     self.feeder_capacity = 2200
90     self.feeder_current_limit = 220
91     self.voltage = 10
92
93     # 故障率
94     self.dg_failure_rate = 0.005
95     self.load_failure_rate = 0.005
96     self.switch_failure_rate = 0.002
97     self.line_failure_rate_per_km = 0.002
98
99     # 记录联络开关信息,包括了联络线的额定功率容量
100     self.tie_switches = {
        "S13-1": {'nodes': (13, 23), 'capacity_kw': 2200},
        "S29-2": {'nodes': (29, 43), 'capacity_kw': 2200},
        "S62-3": {'nodes': (62, 1), 'capacity_kw': 2200}
    }

```

```

101     self.default_tie_line_capacity_kw = 1000
102
103     # 记录变电站出线开关
104     self.substation_switches = {
105         "CB1": 1,
106         "CB2": 23,
107         "CB3": 43
108     }
109
110     # 定义馈线区域
111     self.feeder_regions = {
112         "Feeder1": list(range(1, 23)),
113         "Feeder2": list(range(23, 43)),
114         "Feeder3": list(range(43, 63))
115     }
116
117     # CDF 参数
118     self.interruption_duration_hours = 1.0
119     self.use_linear_cdf = True # 是否启用CDF计算
120     self.linear_cdf_weights = { # 元/kW for the specified duration (e.
121     g., 1 hour)
122         "居民": 1.0, "商业": 5.0, "政府机构": 3.0,
123         "办公建筑": 4.0, "未知类别": 0.5
124     }
125     self.nonlinear_cdf_params = {
126         "居民": {'a': 0.5, 'b': 1.2, 'c': 10, 't_crit': 2.0},
127         "商业": {'a': 2.0, 'b': 1.1, 'c': 50, 't_crit': 1.0},
128         "政府机构": {'a': 1.0, 'b': 1.0, 'c': 30, 't_crit': 1.5},
129         "办公建筑": {'a': 1.5, 'b': 1.15, 'c': 40, 't_crit': 1.0},
130         "未知类别": {'a': 0.1, 'b': 1.0, 'c': 1, 't_crit': 4.0},
131     }
132
133     # 过负荷参数设定
134     self.dg_curtailment_cost_per_kw = 0.2 # 元/kW (示例)
135     self.line_rated_current_a = 220.0 # A, 线路额定电流
136     self.line_overload_threshold_factor = 1.1 # 过载电流
137     self.line_overload_cost_coefficient_ck = 1.0 # 示例 c_k for
138     overload consequence
139     self.power_factor_assumed = 0.95 # 用于标准电流计算 (如果需要)
140
141     self.G = self._build_network()

```

```

141     print("网络模型初始化完成 (含节点分类和风险参数)。")
142
143
144
145     def _build_network(self):
146         """ 构建配电网网络图 """
147         G = nx.Graph()
148
149         for i in range(len(self.nodes_data)):
150             node_id = i + 1
151             node_category = self.node_categories_map.get(node_id, "未知类别")
152             # 获取节点类别
153             G.add_node(node_id,
154                        load=self.nodes_data.loc[i, '有功P/kW'],
155                        category=node_category) # 添加类别属性
156
157         for index, row in self.lines_data.iterrows():
158
159             start_node = int(row['起点'])
160             end_node = int(row['终点'])
161             length = float(row['长度/km'])
162             resistance = float(row['电阻/ $\Omega$ '])
163             reactance = float(row['电抗/ $\Omega$ '])
164
165
166             G.add_edge(start_node, end_node,
167                        id=row.get('编号', f'L{index}'),
168                        length=length,
169                        resistance_total=resistance,
170                        reactance_total=reactance,
171                        r_ohm_per_km=resistance / length if length > 0 else
172                        np.inf,
173                        x_ohm_per_km=reactance / length if length > 0 else
174                        np.inf,
175                        failure_rate=length * self.line_failure_rate_per_km)
176
177             return G
178
179     def _get_feeder_for_node(self, node_id):
180         for name, nodes in self.feeder_regions.items():
181             if node_id in nodes: return name

```

```

180         return None
181
182     def _get_substation_connection_node_for_feeder(self, feeder_name):
183         mapping = {"Feeder1": "CB1", "Feeder2": "CB2", "Feeder3": "CB3"}
184         cb_name = mapping.get(feeder_name)
185         return self.substation_switches.get(cb_name) if cb_name else None
186
187     def _get_feeder_current_status(self, feeder_name, current_G,
188 failed_nodes_set, active_dgs_scenario_capacity):
189         """ 计算电流负载, DG, 以及一条馈线的可用容量. """
190         feeder_total_load_kw = 0
191         feeder_active_dg_kw = 0
192         if feeder_name not in self.feeder_regions: return 0, 0, 0 # Load,
193 DG, T_available
194
195         for node_id in self.feeder_regions[feeder_name]:
196             if current_G.has_node(node_id) and node_id not in
197 failed_nodes_set:
198                 feeder_total_load_kw += self.G.nodes[node_id].get('load',
199 0) # Use original load for demand
200                 # Check DG contribution from active DGs
201                 for dg_id, dg_info in self.dg_data.items():
202                     if dg_info['node'] == node_id and
203 active_dgs_scenario_capacity.get(dg_id, 0) > 0:
204                         feeder_active_dg_kw += active_dgs_scenario_capacity
205 [dg_id]
206
207         net_load_on_feeder = feeder_total_load_kw - feeder_active_dg_kw
208         t_available = 0
209         if net_load_on_feeder >= 0: # Net load or load equals DG
210             t_available = self.feeder_capacity - net_load_on_feeder
211         else: # Net generation on this feeder (DG > Load)
212             t_available = self.feeder_capacity # Can supply up to its
213 rated capacity to others
214
215         return feeder_total_load_kw, feeder_active_dg_kw, max(0,
216 t_available)
217
218     def _calculate_scenario_consequence(self, final_unserved_loads_per_node
219 ):
220         scenario_total_cost = 0
221         for node_id, unserved_kw in final_unserved_loads_per_node.items():

```

```

213         if unserved_kw > 0 and self.G.has_node(node_id): # Ensure node
exists in original graph for category
214             category = self.G.nodes[node_id].get('category', "未知类别"
)
215             Lj = unserved_kw
216             tj = self.interruption_duration_hours
217
218             if self.use_linear_cdf:
219                 Wj = self.linear_cdf_weights.get(category, self.
linear_cdf_weights["未知类别"])
220                 scenario_total_cost += Wj * Lj
221             else:
222                 params = self.nonlinear_cdf_params.get(category, self.
nonlinear_cdf_params["未知类别"])
223                 aj, bj, cj, t_crit_j = params['a'], params['b'], params
['c'], params['t_crit']
224                 cost_at_node = Lj * (aj * (tj ** bj) + cj * (1 if tj >
t_crit_j else 0))
225                 scenario_total_cost += cost_at_node
226             return scenario_total_cost
227
228     def calculate_load_loss_risk(self):
229         """
230         计算失负荷风险（基于蒙特卡洛模拟）
231         R_LL = E[C_LL,scenario] = (1/N_sim) * sum(C_LL,i for i in N_sim)
232         """
233         total_scenario_consequences_sum = 0.0
234
235         for _ in range(self.n_simulations):
236             current_G_scenario = self.G.copy() # 本场景的图表
237
238             # --- 1. 模拟组件故障 ---
239             active_dgs_this_scenario = {} # {dg_id: capacity_kw if active
else 0}
240             for dg_id, dg_info in self.dg_data.items():
241                 if random.uniform(0, 1) < self.dg_failure_rate:
242                     active_dgs_this_scenario[dg_id] = 0 # DG failed
243                 else:
244                     active_dgs_this_scenario[dg_id] = dg_info['capacity']
245
246             failed_lines_this_scenario = []
247             for u, v, data in self.G.edges(data=True): # 遍历图以获取故障

```

率

```
248         if random.uniform(0, 1) < data['failure_rate']:
249             failed_lines_this_scenario.append((u, v))
250         for u, v in failed_lines_this_scenario: # 应用于情景图
251             if current_G_scenario.has_edge(u, v): current_G_scenario.
remove_edge(u, v)

252
253         failed_tie_switches_this_scenario = set()
254         for ts_name in self.tie_switches.keys():
255             if random.uniform(0, 1) < self.switch_failure_rate:
256                 failed_tie_switches_this_scenario.add(ts_name)
257
258         failed_substation_switches_this_scenario = set()
259         for cb_name in self.substation_switches.keys():
260             if random.uniform(0, 1) < self.switch_failure_rate:
261                 failed_substation_switches_this_scenario.add(cb_name)
262
263         failed_load_nodes_this_scenario = set()
264         for node_id in self.G.nodes():
265             is_dg_node = any(dg_info['node'] == node_id for dg_info in
self.dg_data.values())
266             if not is_dg_node: # 非dg的负载消耗节点
267                 if random.uniform(0, 1) < self.load_failure_rate:
268                     failed_load_nodes_this_scenario.add(node_id)
269
270         # --- 2. 网络拓扑分析和初始负载损耗 ---
271         active_substation_sources = {self.substation_switches[cb] for
cb in self.substation_switches
272                                     if cb not in
failed_substation_switches_this_scenario
273                                     and current_G_scenario.has_node(
self.substation_switches[cb])}
274
275         if not active_substation_sources: # Total blackout if all
sources fail
276             unserved_loads_dict = {nid: data['load'] for nid, data in
self.G.nodes(data=True)
277                                     if nid not in
failed_load_nodes_this_scenario and data.get('load', 0) > 0}
278             total_scenario_consequences_sum += self.
_calculate_scenario_consequence(unserved_loads_dict)
279             continue
```



```

280
281     components = list(nx.connected_components(current_G_scenario))
282     scenario_initial_total_unserved_kw = 0
283     islands_requiring_supply = [] # [{'nodes': set, '
initial_unserved_kw': float, 'original_loads_in_island': dict}]
284
285     main_grid_nodes_connected_to_substation = set()
286     for comp in components:
287         if any(src_node in comp for src_node in
active_substation_sources):
288             main_grid_nodes_connected_to_substation.update(comp)
289
290     for comp_nodes in components:
291         is_part_of_main_grid = any(node in
main_grid_nodes_connected_to_substation for node in comp_nodes)
292
293         if not is_part_of_main_grid: # This component is an island
294             island_load_demand_kw = 0
295             island_active_dg_supply_kw = 0
296             original_loads_map_in_island = {}
297
298             for node_id in comp_nodes:
299                 if node_id in failed_load_nodes_this_scenario:
continue # Failed load node has no demand
300
301                 original_load = self.G.nodes[node_id].get('load',
0)
302
303                 if original_load > 0:
304                     island_load_demand_kw += original_load
305                     original_loads_map_in_island[node_id] =
original_load
306
307                 for dg_id, dg_info in self.dg_data.items():
308                     if dg_info['node'] == node_id and
active_dgs_this_scenario.get(dg_id, 0) > 0:
309                         island_active_dg_supply_kw +=
active_dgs_this_scenario[dg_id]
310
311                 unserved_in_island = max(0, island_load_demand_kw -
island_active_dg_supply_kw)
312                 if unserved_in_island > 0:
313                     scenario_initial_total_unserved_kw +=

```

```

unserved_in_island
313         islands_requiring_supply.append({
314             'nodes': comp_nodes,
315             'initial_unserved_kw': unserved_in_island,
316             'original_loads_in_island':
original_loads_map_in_island
317         })
318
319         if scenario_initial_total_unserved_kw == 0:
320             total_scenario_consequences_sum += 0.0
321             continue
322
323         # --- 3. Load Transfer via Tie Switches ---
324         # Calculate available capacity from healthy, connected feeders
325         healthy_feeders_supply_capacity = {} # {feeder_name:
T_available_kw}
326         for feeder_name_iter in self.feeder_regions.keys():
327             sub_conn_node = self.
_get_substation_connection_node_for_feeder(feeder_name_iter)
328             if sub_conn_node and sub_conn_node in
main_grid_nodes_connected_to_substation: # Feeder is healthy and
connected
329                 _, _, t_avail = self._get_feeder_current_status(
feeder_name_iter, current_G_scenario,
330
failed_load_nodes_this_scenario,
331
active_dgs_this_scenario)
332                 healthy_feeders_supply_capacity[feeder_name_iter] =
t_avail
333
334                 final_unserved_loads_dict_scenario = {} # {node_id:
unserved_kw}
335                 for island in islands_requiring_supply: # Initialize with all
island loads as unserved
336                     for node_id, load_val in island['original_loads_in_island'
].items():
337                         final_unserved_loads_dict_scenario[node_id] = load_val
338
339                 # Attempt transfer for each island based on its
initial_unserved_kw
340                 for island in sorted(islands_requiring_supply, key=lambda x: x[

```

```

'initial_unserved_kw'],
reverse=True): # Prioritize larger
unserved
    l_fault_for_this_island = island['initial_unserved_kw']
    if l_fault_for_this_island <= 0: continue

    possible_transfers = [] # [{'from_feeder': str, '
T_avail_at_source_feeder': float, 'tie_capacity': float, 'tie_name': str
}]

    for ts_name, ts_data in self.tie_switches.items():
        if ts_name in failed_tie_switches_this_scenario:
continue

        n1, n2 = ts_data['nodes']
        island_node, healthy_main_grid_node = None, None
        if n1 in island['nodes'] and n2 in
main_grid_nodes_connected_to_substation:
            island_node, healthy_main_grid_node = n1, n2
        elif n2 in island['nodes'] and n1 in
main_grid_nodes_connected_to_substation:
            island_node, healthy_main_grid_node = n2, n1

        if healthy_main_grid_node:
            source_feeder_name = self._get_feeder_for_node(
healthy_main_grid_node)
            if source_feeder_name and source_feeder_name in
healthy_feeders_supply_capacity:
                t_avail_from_feeder =
healthy_feeders_supply_capacity[source_feeder_name]
                s_tie_max = ts_data.get('capacity_kw', self.
default_tie_line_capacity_kw)
                if t_avail_from_feeder > 0 and s_tie_max > 0:
                    possible_transfers.append({
                        'from_feeder': source_feeder_name,
                        'T_avail_at_source_feeder':
t_avail_from_feeder,
                        'tie_capacity': s_tie_max,
                        'tie_name': ts_name
                    })

    if not possible_transfers: continue

```

```

371         sum_T_available_via_ties = sum(pt['T_avail_at_source_feeder
'] for pt in
372                                     possible_transfers) # This
should be sum of min(T_avail_feeder, S_tie)
373         # Correct sum_T_available should be sum of what each path
can *actually* offer up to its own limits
374         sum_potential_supply_for_proportional = sum(
375             min(pt['T_avail_at_source_feeder'], pt['tie_capacity'])
for pt in possible_transfers)
376
377         total_transferred_to_this_island = 0
378         if sum_potential_supply_for_proportional > 0:
379             for pt_info in sorted(possible_transfers,
380                                 key=lambda x: min(x['
T_avail_at_source_feeder'], x['tie_capacity']),
381                                 reverse=True): # Prioritize
stronger paths
382                 # Max this path can offer before proportional split
383                 max_this_path_can_give = min(pt_info['
T_avail_at_source_feeder'], pt_info['tie_capacity'])
384
385                 # Proportional allocation based on what *this path*
can give relative to sum of what *all paths* can give
386                 share_of_fault = (
387                                     max_this_path_can_give
/ sum_potential_supply_for_proportional) * l_fault_for_this_island
388
389                 t_actual_for_this_path = min(max_this_path_can_give
, share_of_fault)
390
391                 # Ensure not to transfer more than remaining fault
for the island
392                 t_actual_for_this_path = min(t_actual_for_this_path
,
393
l_fault_for_this_island - total_transferred_to_this_island)
394
395                 if t_actual_for_this_path > 0:
396                     healthy_feeders_supply_capacity[pt_info['
from_feeder']] -= t_actual_for_this_path
397                     total_transferred_to_this_island +=
t_actual_for_this_path

```

```

398
399         # Distribute total_transferred_to_this_island among nodes
in this island
400         if total_transferred_to_this_island > 0:
401             current_total_unserved_in_island = sum(
402                 final_unserved_loads_dict_scenario.get(nid, 0) for
nid in island['nodes'])
403             if current_total_unserved_in_island > 0: # Should be
same as l_fault_for_this_island initially
404                 for node_id_in_island in island[
405                     'original_loads_in_island'].keys(): # Iterate
only nodes that had load
406                     original_node_load = island['
original_loads_in_island'][node_id_in_island]
407                     # Distribute proportionally to original load
that was unserved
408                     # The unserved load for this node is currently
final_unserved_loads_dict_scenario[node_id_in_island]
409                     if final_unserved_loads_dict_scenario.get(
node_id_in_island, 0) > 0:
410                         proportion_of_island_unserved =
final_unserved_loads_dict_scenario[
411                             node_id_in_island] / current_total_unserved_in_island
412                         reduction_for_node =
proportion_of_island_unserved * total_transferred_to_this_island
413                         final_unserved_loads_dict_scenario[
node_id_in_island] -= reduction_for_node
414                         if final_unserved_loads_dict_scenario[
415                             node_id_in_island] < 0.001: #
Threshold for floating point
416                             final_unserved_loads_dict_scenario[
node_id_in_island] = 0.0
417
418         # --- 4. Calculate Consequence for this Scenario ---
419         scenario_consequence = self._calculate_scenario_consequence(
final_unserved_loads_dict_scenario)
420         total_scenario_consequences_sum += scenario_consequence
421
422         # --- Final Risk Calculation ---
423         if self.n_simulations > 0:
424             average_load_loss_risk = total_scenario_consequences_sum / self

```

```

.n_simulations
    else:
        average_load_loss_risk = 0.0

    return average_load_loss_risk

def _get_subtree_net_load(self, graph, root_node, current_node,
visited_dfs, current_G_nodes_data):
    """ Helper for radial power flow: calculates net load of subtree.
    """
    visited_dfs.add(current_node)
    node_data = current_G_nodes_data.get(current_node)
    if not node_data: return 0

    # Net load at this specific node
    subtree_net_load = node_data.get('load', 0) - node_data.get('
p_dg_actual_kw', 0)

    for neighbor in graph.neighbors(current_node):
        # Check if neighbor is part of the same feeder and not visited
        (to avoid going back up)
        # This simple DFS assumes a tree structure for the feeder from
        the root
        if neighbor not in visited_dfs and self._get_feeder_for_node(
neighbor) == self._get_feeder_for_node(
            root_node):
            subtree_net_load += self._get_subtree_net_load(graph,
root_node, neighbor, visited_dfs,
current_G_nodes_data)
    return subtree_net_load

def calculate_overload_risk(self):
    """
    计算过负荷风险（基于蒙特卡洛模拟），结合经济性削减和物理过载。
    返回一个包含详细风险指标的字典。
    """
    total_scenario_curtailment_costs_sum = 0.0
    total_scenario_physical_overload_costs_sum = 0.0
    scenarios_with_any_physical_overload_count = 0

    if not hasattr(self, 'dg_data') or not self.dg_data:

```

```

458         self.dg_data = {}
459
460         # Initialize accumulators for per-component risk metrics
461         aggregated_curtailement_cost_per_feeder = {f_name: 0.0 for f_name in
self.feeder_regions.keys()}
462         aggregated_physical_overload_cost_per_line = {edge_data['id']: 0.0
for u, v, edge_data in
463                                     self.G.edges(data=
True)}}
464         line_overload_counts = {edge_data['id']: 0 for u, v, edge_data in
self.G.edges(data=True)}}
465
466         # Denominator for standard current calculation (Amps if Pk in kW,
Vl in kV)
467         # Not used if using I = 0.1 * Pk(kW)
468         # sqrt3_v_pf = np.sqrt(3) * self.voltage * self.
power_factor_assumed
469         # if abs(sqrt3_v_pf) < 1e-9: sqrt3_v_pf = 1e-9 # Avoid division by
zero
470
471         for i_sim in range(self.n_simulations):
472             # --- Scenario Setup: Node data for this simulation ---
473             current_G_nodes_data_scenario = {
474                 nid: {'load': data.get('load', 0),
475                     'p_dg_nominal_kw': 0.0, # Nominal output based on
random factor
476                     'p_dg_actual_kw': 0.0} # Actual output after
curtailement
477                 for nid, data in self.G.nodes(data=True)
478             }
479
480             # Simulate nominal DG outputs for this scenario
481             scenario_dg_nominal_outputs_map = {} # {dg_id:
nominal_output_kw}
482             for dg_id, dg_config in self.dg_data.items():
483                 # DG出力波动通过蒙特卡洛模拟生成 (e.g., 50-100% of capacity
)
484
485                 # This is P_dg,i in S_i = P_dg,i - L_i
486                 nominal_output = random.uniform(0.5 * dg_config.get('
capacity', 0), dg_config.get('capacity', 0))
487                 scenario_dg_nominal_outputs_map[dg_id] = nominal_output
488                 node_id_for_dg = dg_config.get('node')

```

```

488         if node_id_for_dg in current_G_nodes_data_scenario:
489             current_G_nodes_data_scenario[node_id_for_dg]['
p_dg_nominal_kw'] += nominal_output
490             # Initially, actual is same as nominal, will be reduced
by curtailment
491             current_G_nodes_data_scenario[node_id_for_dg]['
p_dg_actual_kw'] += nominal_output
492
493         # --- Economic Optimization Stage (DG Curtailment) ---
494         scenario_curtailment_cost_this_sim = 0.0
495
496         for feeder_name in self.feeder_regions.keys():
497             feeder_nodes = self.feeder_regions.get(feeder_name, [])
498             if not feeder_nodes: continue
499
500             # 1. Calculate  $S_i = P_{dg,i} - L_i$  for this feeder
501             feeder_total_nominal_dg_kw = 0
502             feeder_total_load_kw = 0
503             for node_id in feeder_nodes:
504                 node_data = current_G_nodes_data_scenario.get(node_id,
{}))
505                 feeder_total_load_kw += node_data.get('load', 0)
506                 feeder_total_nominal_dg_kw += node_data.get('
p_dg_nominal_kw', 0)
507
508             surplus_on_feeder_s_i = feeder_total_nominal_dg_kw -
feeder_total_load_kw
509
510             if surplus_on_feeder_s_i > 0: # Feeder has surplus DG
511                 # 2. Calculate  $P_{exportable,i}$ 
512                 exportable_total_from_feeder_i = 0
513                 for ts_name, ts_config in self.tie_switches.items():
514                     n1_ts, n2_ts = ts_config['nodes']
515                     # Check if this tie switch connects the current
feeder_name to an adjacent one
516                     node_on_this_feeder = None
517                     node_on_adj = None
518                     if n1_ts in feeder_nodes and self.
_get_feeder_for_node(
519                         n2_ts) != feeder_name and self.G.has_node(
n2_ts):
520                         node_on_this_feeder, node_on_adj = n1_ts, n2_ts

```



```

521         elif n2_ts in feeder_nodes and self.
_get_feeder_for_node(
522             n1_ts) != feeder_name and self.G.has_node(
n1_ts):
523             node_on_this_feeder, node_on_adj = n2_ts, n1_ts
524
525             if not node_on_adj: continue # Tie switch not
relevant or adj node invalid
526
527             adj_feeder_name = self._get_feeder_for_node(
node_on_adj)
528             if not adj_feeder_name: continue
529
530             # Adjacent feeder's remaining capacity: 2.2MW - (
Load_k - DG_k)
531             # Load_k and DG_k are for the *adjacent* feeder k
adj_feeder_load_total_kw = 0
532             adj_feeder_nominal_dg_total_kw = 0 # Use nominal
DG of adjacent feeder
533             for adj_node_id in self.feeder_regions.get(
adj_feeder_name, []):
534                 adj_node_data = current_G_nodes_data_scenario.
get(adj_node_id, {})
535                 adj_feeder_load_total_kw += adj_node_data.get('
load', 0)
536                 adj_feeder_nominal_dg_total_kw += adj_node_data
.get('p_dg_nominal_kw',
537
0) # Nominal DG on adj feeder
538
539                 adj_feeder_net_load_for_receiving_calc =
adj_feeder_load_total_kw - adj_feeder_nominal_dg_total_kw
540                 # If adj_feeder_net_load_for_receiving_calc is
negative, it means adj feeder has surplus, cannot receive much.
541                 # If positive, it's a net load.
542                 # Remaining capacity of adjacent feeder to *receive
* power
543                 adj_feeder_receiving_capacity = max(0,
self.
544 feeder_capacity - adj_feeder_net_load_for_receiving_calc)
545
546                 s_k_max_tie_capacity = ts_config.get('capacity_kw',
547

```

```

self.default_tie_line_capacity_kw)
548         can_export_via_this_tie = min(s_k_max_tie_capacity,
adj_feeder_receiving_capacity)
549         exportable_total_from_feeder_i +=
can_export_via_this_tie

550
551         # 3. Calculate P_curtail,i
552         p_curtail_on_feeder_i = max(0, surplus_on_feeder_s_i -
exportable_total_from_feeder_i)
553
554         if p_curtail_on_feeder_i > 0:
555             # 4. Calculate C_OL,i (curtailment cost for this
feeder in this scenario)
556             curtailment_cost_on_feeder_i =
p_curtail_on_feeder_i * self.dg_curtailment_cost_per_kw
557             scenario_curtailment_cost_this_sim +=
curtailment_cost_on_feeder_i
558             aggregated_curtailment_cost_per_feeder[
feeder_name] += curtailment_cost_on_feeder_i #
Accumulate for averaging
559
560
561             # Distribute curtailment P_curtail_on_feeder_i
among DGs on this feeder_name
562             if feeder_total_nominal_dg_kw > 0: # Avoid
division by zero
563                 for node_id_in_f in feeder_nodes:
564                     node_data_in_f =
current_G_nodes_data_scenario.get(node_id_in_f)
565                     if node_data_in_f and node_data_in_f.get('
p_dg_nominal_kw', 0) > 0:
566                         dg_nominal_at_node = node_data_in_f['
p_dg_nominal_kw']
567                         # Proportion of curtailment for this DG
based on its nominal output
568                         curtailment_for_this_dg_at_node = (
569
dg_nominal_at_node / feeder_total_nominal_dg_kw) *
p_curtail_on_feeder_i
570
571                         # Reduce p_dg_actual_kw
572                         current_G_nodes_data_scenario[
node_id_in_f][

```

```

573         'p_dg_actual_kw'] -=
curtailment_for_this_dg_at_node
574         if current_G_nodes_data_scenario[
node_id_in_f]['p_dg_actual_kw'] < 0:
575             current_G_nodes_data_scenario[
node_id_in_f]['p_dg_actual_kw'] = 0.0
576
577         total_scenario_curtailment_costs_sum +=
scenario_curtailment_cost_this_sim
578
579         # --- Physical Safety Verification Phase (using actual DG
outputs after curtailment) ---
580         scenario_physical_overload_cost_this_sim = 0.0
581         is_any_line_overloaded_this_sim = False
582         line_flows_kw_scenario = {}
583
584         for f_name, f_nodes_list in self.feeder_regions.items():
585             root_node = self._get_substation_connection_node_for_feeder
(f_name)
586             if not root_node or not self.G.has_node(root_node):
continue
587             try:
588                 feeder_graph_view = self.G.subgraph(f_nodes_list)
589                 if not feeder_graph_view.has_node(root_node): continue
590                 dfs_tree_edges = list(nx.dfs_edges(feeder_graph_view,
source=root_node))
591                 for u_parent, v_child in dfs_tree_edges:
592                     visited_for_subtree = set()
593                     # _get_subtree_net_load now uses 'p_dg_actual_kw'
from current_G_nodes_data_scenario
594                     power_flow_on_line = self._get_subtree_net_load(
self.G, root_node, v_child,
595                     visited_for_subtree,
596                     current_G_nodes_data_scenario)
597                     line_flows_kw_scenario[tuple(sorted((u_parent,
v_child)))] = abs(power_flow_on_line)
598                     except Exception as e_flow:
599                         pass
600
601         for u, v, edge_data in self.G.edges(data=True):

```

```

602         line_id_tuple_uv = tuple(sorted((u, v)))
603         line_actual_id_str = edge_data['id']
604         p_k_kw = line_flows_kw_scenario.get(line_id_tuple_uv, 0.0)
605
606         if p_k_kw < 0.01: continue
607
608         # Current calculation as per thesis:  $I_k = 100 * P_k$  (MW) =
609         0.1 *  $P_k$  (kW)
610         i_k_amps = 0.1 * p_k_kw
611
612         threshold_current = self.line_overload_threshold_factor *
613         self.line_rated_current_a # 1.1 *  $I_{rated}$ 
614
615         if i_k_amps > threshold_current:
616             is_any_line_overloaded_this_sim = True
617             if line_actual_id_str not in line_overload_counts:
618                 line_overload_counts[line_actual_id_str] = 0
619                 line_overload_counts[line_actual_id_str] += 1
620
621             # Harm:  $C_{OL,k} = c_k * ((I_k - 1.1 * I_{rated}) / (1.1 * I_{rated}))^2$ 
622             relative_excess_over_threshold = (
623                 i_k_amps -
624                 threshold_current) / threshold_current if threshold_current > 0 else (
625                 float('inf') if i_k_amps > 0 else 0)
626             consequence_k = self.line_overload_cost_coefficient_ck
627             * (relative_excess_over_threshold ** 2)
628             scenario_physical_overload_cost_this_sim +=
629             consequence_k
630
631             if line_actual_id_str not in
632             aggregated_physical_overload_cost_per_line:
633                 aggregated_physical_overload_cost_per_line[
634                 line_actual_id_str] = 0.0
635                 aggregated_physical_overload_cost_per_line[
636                 line_actual_id_str] += consequence_k
637
638             total_scenario_physical_overload_costs_sum +=
639             scenario_physical_overload_cost_this_sim
640             if is_any_line_overloaded_this_sim:
641                 scenarios_with_any_physical_overload_count += 1
642

```

```

633     # --- Final Risk Calculation ---
634     avg_curtailment_cost, avg_physical_overload_cost, overall_P_OL =
0.0, 0.0, 0.0
635     avg_curtailment_cost_per_feeder_final = {f: 0.0 for f in self.
feeder_regions.keys()}
636     avg_physical_overload_cost_per_line_final = {edge_data['id']: 0.0
for u, v, edge_data in
637                                     self.G.edges(data=True
)}}
638     line_overload_frequency_final = {edge_data['id']: 0.0 for u, v,
edge_data in self.G.edges(data=True)}
639
640     if self.n_simulations > 0:
641         avg_curtailment_cost = total_scenario_curtailment_costs_sum /
self.n_simulations
642         avg_physical_overload_cost =
total_scenario_physical_overload_costs_sum / self.n_simulations
643         overall_P_OL = scenarios_with_any_physical_overload_count /
self.n_simulations
644
645         avg_curtailment_cost_per_feeder_final = {f: c / self.
n_simulations for f, c in
646 aggregated_curtailment_cost_per_feeder.items()}
647         avg_physical_overload_cost_per_line_final = {l_id: c / self.
n_simulations for l_id, c in
648 aggregated_physical_overload_cost_per_line.items()}
649         line_overload_frequency_final = {l_id: f / self.n_simulations
for l_id, f in line_overload_counts.items()}
650
651     return {
652         'average_total_overload_risk': avg_curtailment_cost +
avg_physical_overload_cost,
653         'average_curtailment_cost': avg_curtailment_cost,
654         'average_physical_overload_cost': avg_physical_overload_cost,
655         'P_OL_system_overload_probability': overall_P_OL,
656         'average_curtailment_cost_per_feeder':
avg_curtailment_cost_per_feeder_final,
657         'average_physical_overload_cost_per_line':
avg_physical_overload_cost_per_line_final,
658         'line_overload_frequency': line_overload_frequency_final

```

```

659     }
660
661     def generate_overload_risk_report(self, overload_risk_results):
662         """
663         生成过负荷风险报告的文本（简化版）。
664         """
665         report_lines = ["=" * 25 + " 过负荷风险分析报告（简化版） " + "=" *
25]
666         report_lines.append(f"总模拟次数: {self.n_simulations}")
667         report_lines.append("-" * 70)
668         report_lines.append(
669             f"平均总物理过载成本（风险）: {overload_risk_results.get('
average_physical_overload_cost', 0.0):.2f}（成本单位/年）"
670         )
671         report_lines.append(
672             f"系统过载概率（P_OL - 至少一条线路过载的场景比例）: {
overload_risk_results.get('P_OL_system_overload_probability', 0.0) *
100:.2f}%")
673         report_lines.append("-" * 70)
674
675         # DG削减部分已移除
676         # report_lines.append("\n--- DG削减风险详情（按馈线） ---")
677         # report_lines.append("（在此简化版计算中未考虑DG削减成本）")
678         # report_lines.append("-" * 70)
679
680         report_lines.append("\n--- 线路物理过载风险详情 ---")
681         cost_per_line = overload_risk_results.get('
average_physical_overload_cost_per_line', {})
682         freq_per_line = overload_risk_results.get('line_overload_frequency'
, {})
683
684         overloaded_lines_details = []
685         for line_id_str, avg_cost in cost_per_line.items():
686             line_display_name = f"线路 {line_id_str}"
687             current_freq = freq_per_line.get(line_id_str, 0)
688             # Display if there's any cost or any overload occurrence
689             if avg_cost > 0.001 or current_freq > 0: # Use a small
threshold for cost display
690                 overloaded_lines_details.append({
691                     'name': line_display_name,
692                     'avg_cost': avg_cost,
693                     'freq': current_freq * 100
694                 })

```

```

694
695         if not overloaded_lines_details:
696             report_lines.append(" 在此模拟中未检测到显著的线路物理过载。")
697         else:
698             # Sort by frequency first, then by average cost for tie-
699             breaking
700             sorted_overloaded_lines = sorted(overloaded_lines_details, key=
701             lambda x: (x['freq'], x['avg_cost']),
702             reverse=True)
703             report_lines.append(" 高风险线路 (按过载频率和平均成本排序):")
704             for detail in sorted_overloaded_lines[:10]: # Display top 10
705 or so
706                 report_lines.append(
707                     f" - {detail['name']}: 平均过载成本 = {detail['
708 avg_cost']:.2f}, 过载频率 = {detail['freq']:.2f}%")
709                 report_lines.append("-" * 70)
710
711                 report_lines.append("\n报告生成完毕。")
712                 return "\n".join(report_lines)
713
714 def generate_comprehensive_risk_report(self, system_risk_results):
715     """
716     生成综合风险报告的文本。
717     """
718     report_lines = ["=" * 30 + " 综合风险评估报告 " + "=" * 30]
719     report_lines.append(f"总模拟次数: {self.n_simulations}")
720     report_lines.append("=" * 80)
721     report_lines.append(
722         f"系统总平均风险成本: {system_risk_results.get('
723 total_system_risk_cost', 0.0):.2f} (成本单位/年)")
724     report_lines.append("=" * 80)
725
726     # --- 失负荷风险部分 ---
727     ll_details = system_risk_results.get('load_loss_details', {})
728     report_lines.append("\n" + "=" * 28 + " 失负荷风险详情 " + "=" *
28)
729
730     if not ll_details or ll_details.get('average_total_load_loss_risk',
-1) < 0: # Check if it was calculated
731         report_lines.append(" 失负荷风险未计算或计算被跳过。")
732     else:
733         report_lines.append(
734             f"平均总失负荷风险成本: {ll_details.get('

```

```

average_total_load_loss_risk', 0.0):.2f} (成本单位/年)")
    report_lines.append(f"平均未服务功率 (EENS proxy): {ll_details.
get('average_EENS_kw', 0.0):.2f} kW")
    # 可以根据 calculate_load_loss_risk 返回的更详细内容在这里添
    加, 例如:
    # report_lines.append(f" - 主要导致失负荷的故障类型频率: ...")
    # report_lines.append(f" - 平均孤岛形成频率: ...")
    report_lines.append("=" * 80)

    # --- 过负荷风险部分 (使用 generate_overload_risk_report 的内容)
    ---
    ol_details = system_risk_results.get('overload_details', {})
    if not ol_details:
        report_lines.append("\n" + "=" * 28 + " 过负荷风险详情 " + "="
* 28)
        report_lines.append(" 过负荷风险未计算或计算被跳过。")
    else:
        # Append the already formatted overload report part
        # Create a temporary string from generate_overload_risk_report
        temp_overload_report = self.generate_overload_risk_report(
ol_details).splitlines()
        # Skip the header and simulation count from the sub-report if
        already printed
        report_lines.extend(temp_overload_report[3:]) # Start after
        the simulation count line of sub-report

        report_lines.append("\n" + "=" * 80)
        report_lines.append("综合报告生成完毕。")
        return "\n".join(report_lines)
def set_dg_data(self, dg_data):
    """
    设置分布式能源数据
    """
    self.dg_data = dg_data
    for dg_id, info in dg_data.items():
        node_id = info.get('node')
        capacity = info.get('capacity')
        if self.G.has_node(node_id):
            if 'dg_capacity' not in self.G.nodes[node_id]:
                self.G.nodes[node_id]['dg_capacity'] = 0
            self.G.nodes[node_id]['dg_capacity'] += capacity
            self.G.nodes[node_id]['is_dg_node'] = True

```



```

763         else:
764             print(f"警告: DG {dg_id} 配置的节点 {node_id} 在网络中不存在。")
765             print("DG数据已设置。")
766
767
768     def calculate_system_risk(self):
769         """
770         计算系统总风险
771         """
772         # 计算失负荷风险
773         load_loss_risk = self.calculate_load_loss_risk()
774
775         # 计算过负荷风险
776         overload_risk = self.calculate_overload_risk()
777
778         # 计算系统总风险
779         system_risk = load_loss_risk + overload_risk['
average_total_overload_risk']
780
781         # 存储详细结果
782         self.detailed_results = {
783             '失负荷风险': load_loss_risk,
784             '过负荷风险': overload_risk,
785             '系统总风险': system_risk
786         }
787
788         # 保存详细结果
789         #self._save_detailed_results()
790
791         return system_risk
792
793     def draw_network_topology(self, save_path="network_topology.png",
risk_visualization_data=None):
794         # (This function remains largely the same, but will use the
simplified risk_visualization_data)
795         if not self.G or self.G.number_of_nodes() == 0: print("网络图为空，
无法绘制。"); return
796         plt.figure(figsize=(20, 16))
797         try:
798             pos = nx.spring_layout(self.G, k=0.7, iterations=80, seed=42)
799         except Exception as e:

```

```

800         print(f"布局出错: {e}. 尝试随机布局。"); pos = nx.random_layout
      (self.G)
801
802         node_colors_map = {"居民": "skyblue", "商业": "lightcoral", "政府机
      构": "lightgoldenrodyellow",
803                             "办公建筑": "plum", "未知类别": "lightgray"}
804         node_colors = []
805         node_sizes = []
806
807         for n in self.G.nodes():
808             is_sub = any(n == cb_n for cb_n in self.substation_switches.
      values())
809             is_dg = self.G.nodes[n].get('is_dg_node', False)
810             if is_sub:
811                 node_colors.append('darkred'); node_sizes.append(900)
812             elif is_dg:
813                 node_colors.append('forestgreen'); node_sizes.append(750)
814             else:
815                 node_colors.append(
816                     node_colors_map.get(self.G.nodes[n].get('category', "未
      知类别"), "lightgray")); node_sizes.append(
817                     600)
818
819         nx.draw_networkx_nodes(self.G, pos, node_color=node_colors,
      node_size=node_sizes, alpha=0.95,
820                                edgecolors='black', linewidths=0.7)
821
822         edge_colors = []
823         edge_widths = []
824         default_edge_color = 'dimgray'
825         default_edge_width = 1.5
826         all_risk_values_for_norm = []
827
828         if risk_visualization_data and '
      average_physical_overload_cost_per_line' in risk_visualization_data:
829             line_risks = risk_visualization_data['
      average_physical_overload_cost_per_line']
830             # Use line_overload_frequency for coloring if cost is often
      zero but frequency is high
831             # Or, use a combined metric. For now, sticking to cost for
      color intensity.
832             all_risk_values_for_norm = [r for r in line_risks.values() if

```

```

833                                     r > 0.001] # Only consider non-
negligible risks for scaling
834
835         norm = None
836         cmap = None
837         if all_risk_values_for_norm:
838             min_risk, max_risk = min(all_risk_values_for_norm), max(
all_risk_values_for_norm)
839             if abs(max_risk - min_risk) < 1e-6: max_risk = min_risk +
0.01 # Avoid min=max for norm
840             norm = mcolors.Normalize(vmin=min_risk, vmax=max_risk, clip
=True)
841             try:
842                 cmap = plt.colormaps.get_cmap('YlOrRd')
843             except AttributeError:
844                 cmap = plt.cm.get_cmap('YlOrRd')
845
846         for u, v, edge_data_nx in self.G.edges(data=True):
847             line_actual_id = edge_data_nx['id']
848             risk_val = line_risks.get(line_actual_id, 0.0)
849             if risk_val > 0.001 and norm and cmap:
850                 edge_colors.append(cmap(norm(risk_val)))
851                 # Make width also proportional to normalized risk or
frequency
852                 freq_val_norm = risk_visualization_data.get('
line_overload_frequency', {}).get(line_actual_id, 0)
853                 edge_widths.append(
854                     1.0 + 3.0 * (norm(risk_val) + freq_val_norm) / 2.0)
855                 # Example combined metric for width
856             else:
857                 edge_colors.append(default_edge_color)
858                 edge_widths.append(default_edge_width)
859             else:
860                 edge_colors = [default_edge_color] * self.G.number_of_edges()
861                 edge_widths = [default_edge_width] * self.G.number_of_edges()
862
863         nx.draw_networkx_edges(self.G, pos, alpha=0.7, edge_color=
edge_colors, width=edge_widths)
864
865         labels = {n: f"{n}\n{d.get('load', 0)}kW" + (f"\nDG:{d.get('
dg_nominal_capacity_kw', 0)}kW" if d.get(
'is_dg_node') else "") + f"\n({d.get('category', 'N/A')})" for

```

```

n, d in self.G.nodes(data=True)}
866     nx.draw_networkx_labels(self.G, pos, labels=labels, font_size=5,
867                             bbox=dict(facecolor='white', alpha=0.7, pad
=0.1, edgecolor='none'))
868
869     plt.title('配电网拓扑结构与过负荷风险可视化（简化版）', fontsize=40)
870     plt.axis('off');
871     plt.tight_layout()
872
873     if risk_visualization_data and all_risk_values_for_norm and cmap
and norm:
874         sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
875         sm.set_array([])
876         cbar = plt.colorbar(sm, orientation="horizontal", fraction
=0.03, pad=0.02, aspect=30)
877         cbar.set_label('线路平均物理过载成本风险（成本单位/年）',
fontsize=10)
878         cbar.ax.tick_params(labelsize=8)
879
880     try:
881         folder = os.path.dirname(save_path)
882         if folder and not os.path.exists(folder): os.makedirs(folder)
883         plt.savefig(save_path, dpi=300, bbox_inches='tight')
884         print(f"网络拓扑图已保存到：{save_path}")
885     except Exception as e:
886         print(f"保存图像失败：{e}")
887     # plt.show()
888
889
890 # --- 模型使用示例 ---
891 if __name__ == "__main__":
892     excel_file_path = "A题附件.xlsx"
893     nodes_df, lines_df = load_data_from_excel(excel_file_path)
894
895     if nodes_df is not None and lines_df is not None:
896         # 临时的DistributionNetworkRiskModel实例以获取feeder_regions配置
897         _temp_model = DistributionNetworkRiskModel(nodes_df, lines_df)
898         max_node_in_feeders = 0
899         if _temp_model.feeder_regions: # 确保feeder_regions不为空
900             for region_nodes in _temp_model.feeder_regions.values():
901                 if region_nodes: # 确保region_nodes列表不为空
902                     max_node_in_feeders = max(max_node_in_feeders, max(

```

```

region_nodes))

903
904     if len(nodes_df) < max_node_in_feeders:
905         print(
906             f"警告：从Excel加载的负荷数据只有 {len(nodes_df)} 行，但馈
线区域定义涉及到了节点 {max_node_in_feeders}。")
907         missing_rows = max_node_in_feeders - len(nodes_df)
908         if missing_rows > 0:
909             print(f"为缺失的 {missing_rows} 个节点补充默认负荷为0。")
910             nodes_df = pd.concat([nodes_df, pd.DataFrame({'有功P/kW':
[0] * missing_rows})], ignore_index=True)
911
912         model = DistributionNetworkRiskModel(nodes_df, lines_df,
n_simulations=1000) # 主要的实例化
913
914         dg_example_data = {
915             "DG1": {'node': 16, 'capacity': 300},
916             "DG2": {'node': 22, 'capacity': 300},
917             "DG3": {'node': 32, 'capacity': 300},
918             "DG4": {'node': 35, 'capacity': 300},
919             "DG5": {'node': 39, 'capacity': 300},
920             "DG6": {'node': 48, 'capacity': 300},
921             "DG7": {'node': 52, 'capacity': 300},
922             "DG8": {'node': 55, 'capacity': 300},
923
924         }
925         model.set_dg_data(dg_example_data)
926
927         results_folder = "problem1_results_risk_calc"
928         if not os.path.exists(results_folder): os.makedirs(results_folder)
929
930         model.draw_network_topology(save_path=os.path.join(results_folder,
"network_topology_for_risk.png"))
931
932         print("\n开始计算失负荷风险...")
933         load_loss_risk_result = model.calculate_load_loss_risk()
934         print(f"计算得到的平均失负荷风险 (R_LL): {load_loss_risk_result:.2f
} (成本单位/年)")
935
936         print("\n开始计算过负荷风险...")
937         overload_risk_result = model.calculate_overload_risk()
938         print(f"计算得到的平均过负荷风险 (R_OL): {overload_risk_result['

```

```

939     average_total_overload_risk']:.2f} (成本单位/年)")
940
941     print("\n生成过负荷风险报告...")
942     report_content = model.generate_overload_risk_report(
943         overload_risk_result)
944     report_file_path = os.path.join(results_folder, "
945     overload_risk_report.txt")
946     with open(report_file_path, "w", encoding="utf-8") as f_report:
947         f_report.write(report_content)
948     print(f"过负荷风险报告已保存到: {report_file_path}")
949     # print("\n--- 报告内容 ---")
950     # print(report_content) # Optionally print report to console
951
952     print(f"系统总风险为{load_loss_risk_result+overload_risk_result['
953     average_total_overload_risk']:.2f}")
954     print("\n生成风险可视化拓扑图...")
955     model.draw_network_topology(
956         save_path=os.path.join(results_folder, "
957     network_topology_with_overload_risk.png"),
958         risk_visualization_data=overload_risk_result
959     )
960
961 else:
962     print("由于数据加载失败，模型初始化和风险计算未能执行。")

```

问题 (2) 代码

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import os
5  import random
6  # 假设 DistributionNetworkRiskModel 类保存在 Q1.py 文件中
7  from Q1 import DistributionNetworkRiskModel, load_data_from_excel # 假设
8  load_data_from_excel 也在 Q1.py 中
9  SEED = 42 # 保证结果可复现
10 random.seed(SEED)
11 np.random.seed(SEED)
12 # 中文显示设置
13 try:
14     plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
15     plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

```

```

15 except Exception as e:
16     print(f"设置中文字体失败（不影响核心计算）：{e}")
17
18
19 def save_results_to_txt(results_data, filename, folder_name="results"):
20     """
21     保存结果到txt文件，并组织在独立文件夹中。
22     参数：
23     results_data: 要保存的结果数据（可以是DataFrame, dict, 或其他可转换为
24     str的对象）
25     filename: 文件名（例如 "analysis_summary.txt"）
26     folder_name: 文件夹名称（例如 "问题2结果"）
27     """
28     if not os.path.exists(folder_name):
29         os.makedirs(folder_name)
30     filepath = os.path.join(folder_name, filename)
31     with open(filepath, 'w', encoding='utf-8') as f:
32         if isinstance(results_data, pd.DataFrame):
33             f.write(results_data.to_string())
34         elif isinstance(results_data, dict):
35             for key, value in results_data.items():
36                 f.write(f"{key}: {value}\n")
37         else:
38             f.write(str(results_data))
39     print(f"结果已保存到：{filepath}")
40
41 def analyze_dg_capacity_impact(excel_file_path="A题附件.xlsx",
42                                results_base_folder="问题2结果"):
43     """
44     分析分布式能源容量对系统风险的影响
45     以0.3I为步长，从初始容量I增加到3I
46     """
47     nodes_df_from_excel, lines_df_from_excel = None, None
48     if callable(load_data_from_excel) and \
49         load_data_from_excel.__code__.co_code != compile("<
50 string>", "exec").co_code:
51         try:
52             nodes_df_from_excel, lines_df_from_excel = load_data_from_excel
53             (excel_file_path)
54             if nodes_df_from_excel is None or lines_df_from_excel is None:
55                 print("从Excel加载数据失败，将使用内置的示例数据。")

```

```

53         raise FileNotFoundError
54     except Exception as e:
55         print(f"从Excel文件 '{excel_file_path}' 加载数据时出错: {e}。将
使用内置示例数据。")
56         nodes_df_from_excel, lines_df_from_excel = None, None # Ensure
fallback
57     else:
58         print("load_data_from_excel 函数为 pass 或不可调用, 将使用内置示例
数据。")
59
60
61     # 初始化模型时, 确保节点数量与feeder_regions定义匹配
62     # (此逻辑应在 DistributionNetworkRiskModel 的 __init__ 或数据加载后执行
更佳)
63     try:
64         # 创建临时模型以访问feeder_regions (这会导致打印"网络模型初始化完成
"两次)
65         temp_model_for_config = DistributionNetworkRiskModel(
nodes_df_from_excel, lines_df_from_excel, n_simulations=1)
66         max_node_in_feeders = 0
67         if temp_model_for_config.feeder_regions:
68             for region_nodes in temp_model_for_config.feeder_regions.values
():
69                 if region_nodes:
70                     max_node_in_feeders = max(max_node_in_feeders, max(
region_nodes))
71
72         if len(nodes_df_from_excel) < max_node_in_feeders:
73             print(
74                 f"警告: 节点数据行数 {len(nodes_df_from_excel)} 少于馈线定
义最大节点 {max_node_in_feeders}。将补充缺失节点。")
75             nodes_df_from_excel = pd.concat([
76                 nodes_df_from_excel,
77                 pd.DataFrame({'有功P/kW': [0] * (max_node_in_feeders - len(
nodes_df_from_excel))})
78                 ], ignore_index=True)
79     except Exception as e_temp_model:
80         print(f"创建临时模型检查节点数量时出错: {e_temp_model}。可能影响后
续节点相关的操作。")
81
82     # 使用可能已调整的 nodes_df_from_excel 初始化主模型
83     model = DistributionNetworkRiskModel(nodes_df_from_excel,

```



```

lines_df_from_excel, n_simulations=10000)

# 初始DG容量 I (单个DG点的基准容量)
base_capacity_per_dg_unit = 300 # kW

# 设置分布式能源位置
dg_locations_config = { # DG ID: {'node': node_id}
    "DG1": {'node': 13}, "DG2": {'node': 18},
    "DG3": {'node': 22}, "DG4": {'node': 29},
    "DG5": {'node': 32}, "DG6": {'node': 39},
    "DG7": {'node': 48}, "DG8": {'node': 59}
}

# 容量因子范围
capacity_factors = [1.0, 1.3, 1.6, 1.9, 2.2, 2.5, 2.8, 3.0]

results_log = {
    'capacity_factor': [], 'total_dg_capacity_kw': [],
    'load_loss_risk_cost': [], 'overload_risk_cost': [],
    'system_risk_cost': []
}

print("\n开始分析不同DG容量下的系统风险演变...")
for factor in capacity_factors:
    current_total_dg_capacity_all_units = 0
    dg_data_for_model_this_factor = {} # 构建当前因子下的DG配置
    for dg_id, loc_info in dg_locations_config.items():
        current_dg_unit_capacity = base_capacity_per_dg_unit * factor
        dg_data_for_model_this_factor[dg_id] = {
            'node': loc_info['node'],
            'capacity': current_dg_unit_capacity
        }
        current_total_dg_capacity_all_units += current_dg_unit_capacity

    print(f"\n--- 计算容量因子: {factor:.1f} (总DG容量: {
current_total_dg_capacity_all_units:.0f} kW) ---")

    # 设置当前容量的DG数据到模型实例
    # 假设 model.set_dg_data 会正确更新模型内部的DG配置
    if callable(model.set_dg_data) and \
        model.set_dg_data.__code__.co_code != compile("pass", "<
string>", "exec").co_code:

```

```

123     model.set_dg_data(dg_data_for_model_this_factor)
124     else: # 如果 set_dg_data 是 pass, 则手动设置 model.dg_data_config
125         model.dg_data_config = dg_data_for_model_this_factor
126         print("提示: model.set_dg_data 为 pass, 已手动设置 model.
dg_data_config")
127         # 需要确保模型内部的图节点DG信息也得到更新 (如果set_dg_data原本
负责此事)
128         for dg_id_iter, info_iter in model.dg_data_config.items():
129             node_id_iter, capacity_iter = info_iter.get('node'),
info_iter.get('capacity')
130             if model.G.has_node(node_id_iter):
131                 model.G.nodes[node_id_iter]['dg_nominal_capacity_kw'] =
model.G.nodes[node_id_iter].get(
132                     'dg_nominal_capacity_kw', 0) + capacity_iter # 或
者直接设为capacity_iter, 取决于set_dg_data的逻辑
133                 model.G.nodes[node_id_iter]['is_dg_node'] = True
134
135         # 调用模型中的风险计算方法
136
137         # ll_risk_value 是一个浮点数
138         # ol_risk_details 是一个字典
139         # system_risk_details 是一个字典, 包含了ll和ol的详细结果及总和
140
141         # 与调用结构一致
142         ll_risk_value = model.calculate_load_loss_risk()
143         ol_risk_details = model.calculate_overload_risk()
144
145         # 从详细结果中提取用于记录和打印的数值
146         current_ll_risk_cost = ll_risk_value if isinstance(ll_risk_value, (
float, int)) else 0.0
147         current_ol_risk_cost = ol_risk_details.get('
average_total_overload_risk', 0.0) if isinstance(ol_risk_details,
148             dict) else 0.0
149
150         # 计算当前的总系统风险成本
151         current_system_risk_cost = current_ll_risk_cost +
current_ol_risk_cost
152
153         # (或者, 如果Q1.py中的calculate_system_risk已正确实现并考虑了上述返
回类型)
154         # system_risk_details = model.calculate_system_risk()

```

```

155     # current_ll_risk_cost = system_risk_details.get('load_loss_details
156     ', {}).get('average_total_load_loss_risk', 0.0)
157     # current_ol_risk_cost = system_risk_details.get('overload_details
158     ', {}).get('average_total_overload_risk', 0.0)
159     # current_system_risk_cost = system_risk_details.get('
160     total_system_risk_cost', current_ll_risk_cost + current_ol_risk_cost)
161
162     # 记录结果
163     results_log['capacity_factor'].append(factor)
164     results_log['total_dg_capacity_kw'].append(
165     current_total_dg_capacity_all_units)
166     results_log['load_loss_risk_cost'].append(current_ll_risk_cost)
167     results_log['overload_risk_cost'].append(current_ol_risk_cost)
168     results_log['system_risk_cost'].append(current_system_risk_cost)
169
170     print(f" 失负荷风险成本: {current_ll_risk_cost:.2f}")
171     print(f" 过负荷风险成本: {current_ol_risk_cost:.2f}")
172     print(f" 系统总风险成本: {current_system_risk_cost:.2f}")
173
174     results_df = pd.DataFrame(results_log)
175
176     if not os.path.exists(results_base_folder):
177         os.makedirs(results_base_folder)
178
179     results_df.to_csv(os.path.join(results_base_folder, '
180     dg_capacity_risk_evolution.csv'), index=False)
181     print(f"\n风险演变结果已保存到: {os.path.join(results_base_folder, '
182     dg_capacity_risk_evolution.csv')}")
183
184     report_summary_lines = [
185         "=== 分布式能源容量对系统风险的影响分析报告 ===",
186         f"\n1. 基本参数:",
187         f"    - 单个DG点初始容量 (I): {base_capacity_per_dg_unit} kW",
188         f"    - DG数量: {len(dg_locations_config)}",
189         f"    - 容量变化因子范围: {capacity_factors[0]:.1f} 到 {
190     capacity_factors[-1]:.1f}",
191         f"\n2. 风险分析结果汇总:",
192         results_df.to_string(),
193     ]
194
195     if not results_df.empty and 'system_risk_cost' in results_df.columns
196     and results_df[
197         'system_risk_cost'].notna().any():

```

```

189     min_risk_idx = results_df['system_risk_cost'].idxmin()
190     min_risk_row = results_df.loc[min_risk_idx]
191     report_summary_lines.extend([
192         f"\n3. 分析结论:",
193         f"    - 最优容量因子 (系统风险最低): {min_risk_row['
capacity_factor']:.1f}",
194         f"    - 对应总DG容量: {min_risk_row['total_dg_capacity_kw']:.0f}
kW",
195         f"    - 最小系统风险成本: {min_risk_row['system_risk_cost']:.2f}
",
196         f"    - 对应失负荷风险成本: {min_risk_row['load_loss_risk_cost
']:.2f}",
197         f"    - 对应过负荷风险成本: {min_risk_row['overload_risk_cost
']:.2f}",
198     ])
199     else:
200         report_summary_lines.append("\n3. 分析结论: 未能确定最优风险点 (结
果为空或缺少有效风险数据)。")
201
202     save_results_to_txt("\n".join(report_summary_lines), '
dg_capacity_risk_analysis_summary.txt',
203                         folder_name=results_base_folder)
204
205     plt.figure(figsize=(12, 8))
206     plt.plot(results_log['capacity_factor'], results_log['
load_loss_risk_cost'], 'b-o', label='失负荷风险成本')
207     plt.plot(results_log['capacity_factor'], results_log['
overload_risk_cost'], 'r-s', label='过负荷风险成本')
208     plt.plot(results_log['capacity_factor'], results_log['system_risk_cost'
], 'g-^', label='系统总风险成本')
209     plt.xlabel('DG容量因子 (I = 初始容量)')
210     plt.ylabel('风险成本值')
211     plt.title('分布式能源容量对配电系统风险成本的影响演变')
212     plt.grid(True)
213     plt.legend()
214     plt.xticks(capacity_factors)
215     plt.savefig(os.path.join(results_base_folder, '
dg_capacity_risk_evolution_plot.png'), dpi=300)
216     print(f"风险演变图已保存到: {os.path.join(results_base_folder, '
dg_capacity_risk_evolution_plot.png')}")
217     plt.close()
218

```

```

219 # (可选) 调用 analyze_feeder_risks
220 # print("\n开始分析各馈线风险...")
221 # analyze_feeder_risks(model, dg_locations_config,
base_capacity_per_dg_unit, capacity_factors, results_base_folder)
222
223 return results_df
224
225
226 def analyze_feeder_risks(model, dg_locations, base_capacity,
capacity_factors, results_base_folder="问题2结果"):
227     """
228     (占位符/简化) 分析不同DG容量下各馈线的风险变化。
229     """
230     print("警告: analyze_feeder_risks 函数中的馈线风险分配是简化的, 仅用于
演示结构。")
231     feeder_risks_log = {
232         'capacity_factor': [], 'feeder1_risk_placeholder': [],
233         'feeder2_risk_placeholder': [], 'feeder3_risk_placeholder': []
234     }
235
236     for factor in capacity_factors:
237         dg_data_for_model = {}
238         for dg_id, loc_info in dg_locations.items():
239             dg_data_for_model[dg_id] = {'node': loc_info['node'], 'capacity
': base_capacity * factor}
240
241         if callable(model.set_dg_data) and \
242             model.set_dg_data.__code__.co_code != compile("pass", "<
string>", "exec").co_code:
243             model.set_dg_data(dg_data_for_model)
244         else:
245             model.dg_data_config = dg_data_for_model
246             # Manually update graph if set_dg_data is pass
247             for dg_id_iter, info_iter in model.dg_data_config.items():
248                 node_id_iter, capacity_iter = info_iter.get('node'),
info_iter.get('capacity')
249                 if model.G.has_node(node_id_iter):
250                     model.G.nodes[node_id_iter]['dg_nominal_capacity_kw'] =
model.G.nodes[node_id_iter].get(
251                         'dg_nominal_capacity_kw', 0) + capacity_iter
252                     model.G.nodes[node_id_iter]['is_dg_node'] = True
253

```

```

254     system_risk_results = model.calculate_system_risk()
255     total_system_risk = system_risk_results.get('total_system_risk_cost
', 0.0)

256
257     f1_risk = total_system_risk * 0.40
258     f2_risk = total_system_risk * 0.35
259     f3_risk = total_system_risk * 0.25
260
261     feeder_risks_log['capacity_factor'].append(factor)
262     feeder_risks_log['feeder1_risk_placeholder'].append(f1_risk)
263     feeder_risks_log['feeder2_risk_placeholder'].append(f2_risk)
264     feeder_risks_log['feeder3_risk_placeholder'].append(f3_risk)
265
266     feeder_risks_df = pd.DataFrame(feeder_risks_log)
267     feeder_risks_df.to_csv(os.path.join(results_base_folder, '
feeder_risk_analysis_simplified.csv'), index=False)
268     save_results_to_txt(feeder_risks_df, 'feeder_risk_analysis_simplified.
txt', folder_name=results_base_folder)
269
270     plt.figure(figsize=(12, 8))
271     plt.plot(feeder_risks_log['capacity_factor'], feeder_risks_log['
feeder1_risk_placeholder'], 'b-o',
272             label='馈线1风险 (占位符)')
273     plt.plot(feeder_risks_log['capacity_factor'], feeder_risks_log['
feeder2_risk_placeholder'], 'r-s',
274             label='馈线2风险 (占位符)')
275     plt.plot(feeder_risks_log['capacity_factor'], feeder_risks_log['
feeder3_risk_placeholder'], 'g-^',
276             label='馈线3风险 (占位符)')
277     plt.xlabel('DG容量因子 (I = 初始容量)');
278     plt.ylabel('风险值 (占位符)')
279     plt.title('分布式能源容量对各馈线风险的影响 (简化占位符)');
280     plt.grid(True);
281     plt.legend()
282     plt.xticks(capacity_factors)
283     plt.savefig(os.path.join(results_base_folder, '
feeder_risk_analysis_simplified_plot.png'), dpi=300)
284     plt.close()
285     print(f"馈线风险分析 (简化版) 结果已保存到 '{results_base_folder}' 文件
夹。")
286
287

```

```

288 if __name__ == "__main__":
289     excel_data_file = "A题附件.xlsx"
290     main_results_folder = "问题2分析结果"
291     results_dataframe = analyze_dg_capacity_impact(excel_file_path=
excel_data_file,
292                                                     results_base_folder=
main_results_folder)
293
294     if results_dataframe is not None:
295         print("\nDG容量影响分析完成。结果摘要:")
296         print(results_dataframe[['capacity_factor', 'total_dg_capacity_kw',
'system_risk_cost']])
297     else:
298         print("\nDG容量影响分析未能生成有效结果。")

```

模型灵敏度分析代码

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import os
5 import random
6 # 假设 DistributionNetworkRiskModel 类保存在 Q1.py 文件中
7 # 并假设 load_data_from_excel 也在其中或可被导入
8 from Q1 import DistributionNetworkRiskModel, load_data_from_excel
9
10 # 中文显示设置
11 try:
12     plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
13     plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
14 except Exception as e:
15     print(f"设置中文字体失败（不影响核心计算）：{e}")
16
17
18 def save_results_to_file(data_to_save, filename, folder_name="灵敏度与误差
分析结果", format_type="txt"):
19     """
20     保存结果到文件（txt 或 csv），并组织在独立文件夹中。
21     参数：
22     data_to_save: 要保存的结果数据（可以是DataFrame, dict, 或其他可转换为
str的对象）
23     filename: 文件名（例如 "sensitivity_line_fault_rate.txt"）
24     folder_name: 文件夹名称

```

```

25     format_type: "txt" 或 "csv" (csv仅对DataFrame有效)
26     """
27     if not os.path.exists(folder_name):
28         os.makedirs(folder_name)
29     filepath = os.path.join(folder_name, filename)
30
31     if format_type == "csv" and isinstance(data_to_save, pd.DataFrame):
32         data_to_save.to_csv(filepath, index=False, encoding='utf-8-sig')
33     else: # Default to txt
34         with open(filepath, 'w', encoding='utf-8') as f:
35             if isinstance(data_to_save, pd.DataFrame):
36                 f.write(data_to_save.to_string())
37             elif isinstance(data_to_save, dict):
38                 for key, value in data_to_save.items():
39                     f.write(f"{key}: {value}\n")
40             else:
41                 f.write(str(data_to_save))
42     print(f"结果已保存到: {filepath}")
43
44
45 def run_sensitivity_analysis_on_parameter(
46     base_nodes_df, base_lines_df,
47     dg_locations_config, base_dg_capacity_per_unit,
48     fixed_dg_capacity_factor,
49     parameter_name_to_vary, parameter_values,
50     n_simulations_for_sensitivity, results_base_folder,
51     analysis_label # 用于文件名和图表标题
52 ):
53     """
54     对模型中的单个参数进行灵敏度分析。
55
56     参数:
57     base_nodes_df, base_lines_df: 基础的节点和线路数据。
58     dg_locations_config: DG的位置配置。
59     base_dg_capacity_per_unit: 单个DG点的基础容量 (I)。
60     fixed_dg_capacity_factor: 在本次灵敏度分析中固定的DG容量因子 (例如, 分析其他参数时, DG容量保持在 I 或 2I)。
61     parameter_name_to_vary (str): DistributionNetworkRiskModel 实例中要改变的属性名。
62     parameter_values (list): 要测试的参数值列表。
63     n_simulations_for_sensitivity (int): 每次参数变化时运行的蒙特卡洛模拟次数。

```



```

63 results_base_folder (str): 保存结果的根文件夹。
64 analysis_label (str): 用于标识本次分析的标签。
65 """
66 print(f"\n--- 开始对参数 '{parameter_name_to_vary}' 进行灵敏度分析 ({
analysis_label}) ---")
67
68 results_log = {
69     'parameter_value': [],
70     'load_loss_risk_cost': [],
71     'overload_risk_cost': [], # This will be the total overload risk (
curtailment + physical)
72     'system_risk_cost': []
73 }
74
75 # 准备固定的DG配置 (基于 fixed_dg_capacity_factor)
76 fixed_dg_data_for_model = {}
77 for dg_id, loc_info in dg_locations_config.items():
78     fixed_dg_data_for_model[dg_id] = {
79         'node': loc_info['node'],
80         'capacity': base_dg_capacity_per_unit *
fixed_dg_capacity_factor
81     }
82
83 for p_value in parameter_values:
84     print(f" 测试参数 '{parameter_name_to_vary}' = {p_value}")
85
86     # 每次参数变化时重新实例化模型，以确保参数正确应用
87     # (特别是那些在 __init__ 或 _build_network 中使用的参数)
88     model_instance = DistributionNetworkRiskModel(
89         base_nodes_df.copy(), # 使用副本以防意外修改
90         base_lines_df.copy(),
91         n_simulations=n_simulations_for_sensitivity
92     )
93
94     # 设置/修改被分析的参数
95     if hasattr(model_instance, parameter_name_to_vary):
96         setattr(model_instance, parameter_name_to_vary, p_value)
97         # 如果改变的参数影响网络构建 (如 line_failure_rate_per_km) , 需
要重新构建网络
98         # 当前模型在 __init__ 中调用 _build_network, 所以重新实例化已处理
此情况
99         if parameter_name_to_vary == 'line_failure_rate_per_km': # 特

```

殊处理，因为它影响边的属性

```
100         model_instance.G = model_instance._build_network() # 确保
使用更新后的故障率
101     else:
102         print(f"警告：参数 '{parameter_name_to_vary}' 不是模型已知属
性，跳过此值的设置。")
103         # continue # 或者引发错误
104
105     # 设置固定的DG配置
106     if callable(model_instance.set_dg_data) and \
107         model_instance.set_dg_data.__code__.co_code != compile("
pass", "<string>", "exec").co_code:
108         model_instance.set_dg_data(fixed_dg_data_for_model)
109     else:
110         model_instance.dg_data_config = fixed_dg_data_for_model
111         # 手动更新图节点（如果set_dg_data为pass）
112         for dg_id_iter, info_iter in model_instance.dg_data_config.
items():
113             node_id_iter, capacity_iter = info_iter.get('node'),
info_iter.get('capacity')
114             if model_instance.G.has_node(node_id_iter):
115                 model_instance.G.nodes[node_id_iter]['
dg_nominal_capacity_kw'] = model_instance.G.nodes[
116
117                 node_id_iter].get(
118                     'dg_nominal_capacity_kw', 0) + capacity_iter
119                 model_instance.G.nodes[node_id_iter]['is_dg_node'] =
True
120
121     # 运行风险计算
122     ll_risk_results = model_instance.calculate_load_loss_risk()
123     ol_risk_results = model_instance.calculate_overload_risk()
124
125     current_ll_risk = ll_risk_results.get('average_total_load_loss_risk
', 0.0) if isinstance(ll_risk_results,
126
127                     dict) else (
128         ll_risk_results if isinstance(ll_risk_results, (float, int))
else 0.0)
129     current_ol_risk = ol_risk_results.get('average_total_overload_risk'
, 0.0) if isinstance(ol_risk_results,
```

```

dict) else 0.0

current_sys_risk = current_ll_risk + current_ol_risk

results_log['parameter_value'].append(p_value)
results_log['load_loss_risk_cost'].append(current_ll_risk)
results_log['overload_risk_cost'].append(current_ol_risk)
results_log['system_risk_cost'].append(current_sys_risk)

print(f"    系统风险: {current_sys_risk:.2f} (失负荷: {
current_ll_risk:.2f}, 过负荷: {current_ol_risk:.2f})")

results_df = pd.DataFrame(results_log)

# 保存结果
folder_path = os.path.join(results_base_folder, f"sensitivity_{
parameter_name_to_vary.replace('.', '_')}")
save_results_to_file(results_df, f"sensitivity_results_{analysis_label
}.csv", folder_name=folder_path,
                        format_type="csv")
save_results_to_file(results_df.describe(), f"sensitivity_summary_{
analysis_label}.txt", folder_name=folder_path)

# 绘制结果
plt.figure(figsize=(10, 6))
plt.plot(results_df['parameter_value'], results_df['load_loss_risk_cost
'], 'b-o', label='失负荷风险成本')
plt.plot(results_df['parameter_value'], results_df['overload_risk_cost'
], 'r-s', label='过负荷风险成本')
plt.plot(results_df['parameter_value'], results_df['system_risk_cost'],
'g-^', label='系统总风险成本')
plt.xlabel(f"参数 '{parameter_name_to_vary}' 的值")
plt.ylabel('风险成本值')
plt.title(f"参数 '{parameter_name_to_vary}' 对系统风险的灵敏度 ({
analysis_label})")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(folder_path, f"sensitivity_plot_{
analysis_label}.png"), dpi=300)
plt.close()
print(f"灵敏度分析图表已保存到 '{folder_path}' 文件夹。")
return results_df

```

```

161
162
163 def discuss_error_analysis_approaches(results_base_folder):
164     """
165     讨论误差分析的方法和考虑因素。
166     """
167     discussion = [
168         "=== 误差分析讨论 ===",
169         "误差分析旨在评估模型结果的不确定性和可靠性，其来源可能包括：",
170         "1. 输入数据不确定性:",
171         "    - 故障率：通常是统计估计值，本身带有置信区间。可进行参数扰动分
172         析。",
173         "    - 负荷预测误差：实际负荷与模型输入负荷的偏差。",
174         "    - DG出力预测误差：DG出力受天气等随机因素影响，预测模型存在误
175         差。",
176         "    - 经济参数不确定性：如停电损失成本、削减成本等可能随市场、政策
177         变化。",
178         "    方法：对关键输入参数赋予概率分布（而非点估计），通过多次模拟
179         （如嵌套蒙特卡洛）传播不确定性，得到风险值的概率分布或置信区间。",
180         "",
181         "2. 模型结构不确定性（模型简化带来的误差）:",
182         "    - 潮流计算简化：当前模型采用基于辐射状假设的潮流估算，与精确的
183         交流潮流计算结果存在差异，尤其在环网或复杂网络中。",
184         "    改进：与专业潮流工具（如Pandapower）的结果进行对比验证，评估
185         简化模型的适用范围和误差大小。",
186         "    - 元件故障模型简化：如未考虑故障的多种模式、修复时间分布等。",
187         "    - DG出力模型简化：如未使用更精确的Beta分布、Weibull分布或考虑时
188         序相关性。",
189         "    - 联络线转供逻辑简化：实际转供受更多运行约束和保护策略影响。",
190         "    方法：通过与更详细、更精确的模型进行对比，或针对特定简化假设进
191         行专项分析（例如，比较不同潮流计算方法下的风险差异）。",
192         "",
193         "3. 蒙特卡洛模拟收敛性误差:",
194         "    - 模拟次数不足可能导致结果不稳定，不同运行批次结果差异较大。",
195         "    方法：进行收敛性分析，即逐步增加模拟次数（N_simulations），观察
196         风险均值和方差的变化。当结果稳定在可接受范围内时，认为模拟已收敛。",
197         "    （下面的 'run_simulation_count_stability_analysis' 函数即为此目
198         的）",
199         "",
200         "4. 参数校准误差:",
201         "    - 模型中的许多参数（如故障率、成本系数）需要通过历史数据或行业
202         标准进行校准，校准过程本身可能引入误差。",

```

```

192         "    方法：采用历史数据进行回溯测试（back-testing），评估模型的预测
能力。",
193         "",
194         "误差分析的输出:",
195         "    - 风险值的置信区间或概率分布。",
196         "    - 对模型结果稳健性的评估。",
197         "    - 识别对结果不确定性贡献最大的因素，指导数据采集和模型改进的优
先方向。",
198     ]
199     save_results_to_file("\n".join(discussion), "error_analysis_discussion.
txt", folder_name=results_base_folder)
200
201
202 def run_simulation_count_stability_analysis(
203     base_nodes_df, base_lines_df,
204     dg_locations_config, base_dg_capacity_per_unit,
205     fixed_dg_capacity_factor,
206     simulation_counts, results_base_folder
207 ):
208     """
209     分析不同蒙特卡洛模拟次数对风险结果稳定性的影响。
210     """
211     print("\n--- 开始分析模拟次数对结果稳定性的影响 ---")
212     analysis_label = "n_simulations_stability"
213     results_log = {
214         'n_simulations': [],
215         'system_risk_cost_run1': [],
216         'system_risk_cost_run2': [],
217         'system_risk_cost_run3': [], # 运行3次以观察波动
218         'avg_system_risk': [],
219         'std_dev_system_risk': []
220     }
221
222     fixed_dg_data_for_model = {}
223     for dg_id, loc_info in dg_locations_config.items():
224         fixed_dg_data_for_model[dg_id] = {
225             'node': loc_info['node'],
226             'capacity': base_dg_capacity_per_unit *
fixed_dg_capacity_factor
227         }
228
229     for n_sim in simulation_counts:

```

```

229     print(f"    测试模拟次数: {n_sim}")
230     current_run_risks = []
231     for run_idx in range(3): # Perform 3 independent runs for each
n_sim
232         # 重要的: 确保每次独立运行有不同的随机序列, 除非是想测试种子的
效果
233         # 如果之前设置了全局种子, 这里可能需要重新打乱或不设置
234         # random.seed(None) # 取消固定种子, 或使用不同的种子
235         # np.random.seed(None)
236
237         model_instance = DistributionNetworkRiskModel(
238             base_nodes_df.copy(), base_lines_df.copy(), n_simulations=
int(n_sim) # n_sim must be int
239         )
240         # DG data must be set for each instance
241         if callable(model_instance.set_dg_data) and \
242             model_instance.set_dg_data.__code__.co_code != compile(
"pass", "<string>", "exec").co_code:
243             model_instance.set_dg_data(fixed_dg_data_for_model)
244         else:
245             model_instance.dg_data_config = fixed_dg_data_for_model
246             for dg_id_iter, info_iter in model_instance.dg_data_config.
items():
247                 node_id_iter, capacity_iter = info_iter.get('node'),
info_iter.get('capacity')
248                 if model_instance.G.has_node(node_id_iter):
249                     model_instance.G.nodes[node_id_iter]['
dg_nominal_capacity_kw'] = model_instance.G.nodes[
250
251                         node_id_iter].get(
252                             'dg_nominal_capacity_kw', 0) + capacity_iter
253                         model_instance.G.nodes[node_id_iter]['is_dg_node']
= True
254
255             system_risk_details = model_instance.calculate_system_risk()
256             current_run_risks.append(system_risk_details)
257
258     results_log['n_simulations'].append(n_sim)
259     results_log['system_risk_cost_run1'].append(current_run_risks[0])
260     results_log['system_risk_cost_run2'].append(current_run_risks[1])
261     results_log['system_risk_cost_run3'].append(current_run_risks[2])
262     results_log['avg_system_risk'].append(np.mean(current_run_risks))

```

```

262         results_log['std_dev_system_risk'].append(np.std(current_run_risks)
)
263         print(f"      N_sim={n_sim}: Avg Risk={np.mean(current_run_risks):.2f
}, Std Dev={np.std(current_run_risks):.2f}")
264
265     results_df = pd.DataFrame(results_log)
266     folder_path = os.path.join(results_base_folder, f"stability_{
analysis_label}")
267     save_results_to_file(results_df, f"stability_results_{analysis_label}.
csv", folder_name=folder_path,
268                          format_type="csv")
269
270     plt.figure(figsize=(12, 7))
271     plt.errorbar(results_df['n_simulations'], results_df['avg_system_risk'
],
272                yerr=results_df['std_dev_system_risk'], fmt='-o', capsize
=5,
273                label='平均系统风险（带标准差）')
274     plt.plot(results_df['n_simulations'], results_df['system_risk_cost_run1
'], 'r:x', label='运行1风险')
275     # plt.plot(results_df['n_simulations'], results_df['
system_risk_cost_run2'], 'g--s', label='运行2风险')
276     # plt.plot(results_df['n_simulations'], results_df['
system_risk_cost_run3'], 'm-.^', label='运行3风险')
277     plt.xlabel('蒙特卡洛模拟次数 (N_simulations)')
278     plt.ylabel('系统总风险成本值')
279     plt.title('模拟次数对系统风险结果稳定性的影响')
280     plt.xscale('log') # X轴使用对数刻度可能更好看
281     plt.grid(True, which="both", ls="--")
282     plt.legend()
283     plt.tight_layout()
284     plt.savefig(os.path.join(folder_path, f"stability_plot_{analysis_label
}.png"), dpi=300)
285     plt.close()
286     print(f"模拟次数稳定性分析图表已保存到 '{folder_path}' 文件夹。")
287     return results_df
288
289
290 if __name__ == "__main__":
291     # --- 基本配置 ---
292     excel_data_file = "A题附件.xlsx"
293     main_results_folder = "灵敏度与误差分析结果"

```

```

294     if not os.path.exists(main_results_folder):
295         os.makedirs(main_results_folder)
296
297     # 加载基础数据
298     base_nodes_df, base_lines_df = None, None
299     if callable(load_data_from_excel) and \
300         load_data_from_excel.__code__.co_code != compile("pass", "<
string>", "exec").co_code:
301         try:
302             base_nodes_df, base_lines_df = load_data_from_excel(
303 excel_data_file)
304         except Exception as e_load:
305             print(f"从Excel文件 '{excel_data_file}' 加载数据时出错: {e_load
306 }。")
307
308     if base_nodes_df is None or base_lines_df is None:
309         print(
310             "错误：无法加载基础节点和线路数据，灵敏度分析中止。请确保 'A题
311 附件.xlsx' 存在且格式正确，或者 'load_data_from_excel' 函数已正确实现。"
312 )
313     else:
314         # 确保节点数据行数与馈线区域定义的最大节点号匹配（与Q2分析类似）
315         try:
316             _temp_model_config = DistributionNetworkRiskModel(base_nodes_df
317 , base_lines_df, n_simulations=1)
318             max_node_in_feeders = 0
319             if _temp_model_config.feeder_regions:
320                 for region_nodes in _temp_model_config.feeder_regions.
321 values():
322                     if region_nodes: max_node_in_feeders = max(
323 max_node_in_feeders, max(region_nodes))
324             if len(base_nodes_df) < max_node_in_feeders:
325                 print(
326                     f"警告：负荷数据行数 {len(base_nodes_df)} 少于馈线定义
327 最大节点 {max_node_in_feeders}。补充缺失节点。")
328                 base_nodes_df = pd.concat(
329                     [base_nodes_df, pd.DataFrame({'有功P/kW': [0] * (
330 max_node_in_feeders - len(base_nodes_df))})],
331                     ignore_index=True)
332             except Exception as e_init_temp:
333                 print(f"创建临时模型以检查配置时出错: {e_init_temp}")

```



```

326     # 定义DG配置（与Q2分析类似，用于设定一个基准场景）
327     base_dg_capacity_per_unit_for_sa = 300 # kW
328     fixed_dg_capacity_factor_for_sa = 1.0 # 分析其他参数时，DG容量固定
为 I
329     dg_locations_config_for_sa = {
330         "DG1": {'node': 13}, "DG2": {'node': 18}, "DG3": {'node': 22},
331         "DG4": {'node': 29},
332         "DG5": {'node': 32}, "DG6": {'node': 39}, "DG7": {'node': 48},
333         "DG8": {'node': 59}
334     }
335
336     # --- 运行灵敏度分析 ---
337     # 示例1：对线路单位长度故障率进行灵敏度分析
338     run_sensitivity_analysis_on_parameter(
339         base_nodes_df, base_lines_df,
340         dg_locations_config_for_sa, base_dg_capacity_per_unit_for_sa,
341         fixed_dg_capacity_factor_for_sa,
342         parameter_name_to_vary='line_failure_rate_per_km',
343         parameter_values=np.linspace(0.001, 0.005, 5), # 例如，从0.001
到0.005变化
344         n_simulations_for_sensitivity=500, # 灵敏度分析时用的模拟次
数，可适当减少以加快速度
345         results_base_folder=main_results_folder,
346         analysis_label="线路故障率"
347     )
348
349     # 示例2：对DG削减成本进行灵敏度分析
350     run_sensitivity_analysis_on_parameter(
351         base_nodes_df, base_lines_df,
352         dg_locations_config_for_sa, base_dg_capacity_per_unit_for_sa,
353         fixed_dg_capacity_factor_for_sa,
354         parameter_name_to_vary='dg_curtailment_cost_per_kw',
355         parameter_values=np.array([0.1, 0.2, 0.3, 0.4, 0.5]),
356         n_simulations_for_sensitivity=500,
357         results_base_folder=main_results_folder,
358         analysis_label="DG削减成本"
359     )
360
361     # 示例3：对线路过载成本系数c_k进行灵敏度分析
362     run_sensitivity_analysis_on_parameter(
363         base_nodes_df, base_lines_df,
364         dg_locations_config_for_sa, base_dg_capacity_per_unit_for_sa,

```

```

fixed_dg_capacity_factor_for_sa,
361     parameter_name_to_vary='line_overload_cost_coefficient_ck',
362     parameter_values=np.array([0.5, 1.0, 1.5, 2.0, 2.5]),
363     n_simulations_for_sensitivity=500,
364     results_base_folder=main_results_folder,
365     analysis_label="线路过载成本系数"
366 )
367
368 # --- 误差分析讨论与模拟次数稳定性 ---
369 discuss_error_analysis_approaches(main_results_folder)
370
371 run_simulation_count_stability_analysis(
372     base_nodes_df, base_lines_df,
373     dg_locations_config_for_sa, base_dg_capacity_per_unit_for_sa,
fixed_dg_capacity_factor_for_sa,
374     simulation_counts=[100, 200, 500, 1000, 2000, 5000], # 测试不
同的模拟次数
375     results_base_folder=main_results_folder
376 )
377
378 print("\n灵敏度与误差分析（初步）完成。")

```